

Analyse des stations-service et des prix des carburants à partir de données ouvertes.

Théo Palugan
Loïc Lakshminarayanan
Nathanaël Dos Santos

BUT 1 Informatique

Équipe : Les phacochères pas trop chers

Groupe : Vénus

Année 2025 - 2026

IUT de Villetaneuse (Sorbonne université paris 13)

Base de Données (SAE S204)

11/05/2026 - 7/06/2026

Sommaire :

Sommaire :	2
I - Livrable 1	3
1. Composition de l'équipe	3
2. Organisation du travail	3
3. Outils utilisés	4
4. Planning prévisionnel	4
II - Livrable 2	6
1. Présentation des données.	6
2. Identification des principales entités.	7
3. Relation entre les tables.	7
4. Clés primaires et clés étrangères.	8
5. Schéma relationnel	8
6. Justification des principaux choix de modélisation	9
II - Livrable 3	11
1. Scripts SQL de création des tables (Clés et contraintes) pour PSQL.	11
2. Scripts importation ou alimentation des données.	13
3. Explication mécanisme d'historisation des prix.	25
4. Ensemble des requêtes SQL d'analyse.	26
5. Interprétation des premiers résultats.	38
II - Livrable 4	39
1. Scripts SQL de création des tables (Clés et contraintes) pour PSQL.	39
2. Scripts importation ou alimentation des données.	39
3. Ensemble des requêtes SQL d'analyse.	39
4. Visualisations produites.	40
5. Tableau de bord Grafana.	64
6. Synthèse des résultats obtenus.	66
7. Conclusion sur les principaux enseignements du projet.	68
Annexes :	69

I - Livrable 1

1. Composition de l'équipe

Nom de l'équipe : *Les phacochères pas trop chers*

Membre : Théo Palugan / Loïc Lakshminarayanan / Nathanael Dos Santos

Moyens de communication : Discord et Whatsapp

2. Organisation du travail

Répartition des tâches et des rôles :

Théo Palugan : Responsable base de données et visualisation

Loïc Lakshminarayanan : Responsable modélisation et alimentation des données

Nathanaël Dos Santos : Responsable schéma relationnel et requêtes SQL

Tâches Principales	Théo	Loïc	Nathanael
Planning	X	X	X
Modèle de données		X	
Schéma relationnel			X
Script de création de la base de données	X		
Script d'alimentation de la base		X	
Ensemble de requêtes d'analyse			X
Tableaux de bord de visualisation	X		
Synthèse des résultats	X	X	X

Méthode de travail choisie :

En fonction de la complexité et de la durée des tâches, nous avons décidé de soit travailler chacun une tâche individuellement ou en collectif pour aboutir dans les deux cas à une mise en commun et vérifier que tous les membres de l'équipe ont bien compris les notions. Notre travail sera en hybride, il se fera donc à la fois en distanciel mais également en présentiel.

Nous avons décidé de faire un point au moins une fois par semaine afin de vérifier, les tâches terminées, les différents problèmes rencontrés, d'expliquer ce que nous avons réalisé (si la tâche réalisée était individuelle) ainsi que les tâches à faire pour la semaine suivante.

3. Outils utilisés

Les outils que nous allons utiliser :

- 1 - Discord : Communication du groupe et appelle.
- 2 - Google Drive : Partage et stockage des fichiers.
- 3 - Whatsapp : Échange rapides et coordination.
- 4 - Grafana : Création de tableaux de bord.
- 5 - PostgreSQL : Requêtes SQL
- 6 - Python : Collecte et traitement des données .

4. Planning prévisionnel

Date	Tâches Principales	Objectifs intermédiaires
11/05/2026 au 17/05/2026	Planning	nom de l'équipe membres de l'équipe moyens de communication utilisés répartition initiale des tâches rôle de chaque membre méthode de travail choisie

18/05/2026 au 24/05/2026	Modèle de données	une présentation rapide des données exploitées l'identification des principales entités
18/05/2026 au 24/05/2026	Schéma relationnel	les relations entre les tables les clés primaires et les clés étrangères un MCD, un MLD ou un schéma relationnel une justification des principaux choix de modélisation
25/05/2026 au 31/05/2026	Script de création de la base de données	les scripts SQL de création des tables les clés primaires et les clés étrangères les contraintes d'intégrité
25/05/2026 au 31/05/2026	Script d'alimentation de la base	les scripts d'importation ou d'alimentation des données une explication du mécanisme d'historisation des prix
25/05/2026 au 31/05/2026	Ensemble de requêtes d'analyse	un ensemble de requêtes SQL d'analyse une courte interprétation des premiers résultats obtenus
01/06/2026 au 07/06/2026	Ensemble de requêtes d'analyse	les scripts SQL finalisés les scripts Python utilisés pour le traitement ou l'importation des données les requêtes SQL d'analyse
01/06/2026 au 07/06/2026	Tableaux de bord de visualisation	les visualisations produites un tableau de bord, réalisé par exemple avec Grafana
01/06/2026 au 07/06/2026	Synthèse des résultats	une synthèse des résultats obtenus une conclusion sur les principaux enseignements du projet

Diagramme de Gantt :

En rouge : les livrables (rendus)

En bleu : les tâches

	Semaine	1					2					3					4					5
	Jour	L	M	M	J	V	L	M	M	J	V	L	M	M	J	V	L	M	M	J	V	L
Tâches Principales	DATE (début)																					
Planning	11/05/2026																					
Modèle de données	18/05/2026																					
Schéma relationnel	20/05/2026																					
Script de création de la base de données	25/05/2026																					
Script d'alimentation de la base	27/05/2026																					
Ensemble de requêtes d'analyse	28/05/2026																					
Ensemble de requêtes d'analyse	01/06/2026																					
Tableaux de bord de visualisation	03/06/2026																					
Synthèse des résultats	05/06/2026																					

II - Livrable 2

1. Présentation des données.

Nous allons devoir utiliser différents types de données comme des stations-service pour connaître leur localisation, adresse et enseigne. Mais également les carburants proposés par ces stations comme le gazole, SP95, E10, GPL, ...

De plus nous devrons utiliser des données qui nous permettront de trouver le prix associé à chaque carburant mais également les dates de mise à jour des prix et des éventuelles ruptures.

Mais encore les horaires d'ouverture des stations et les services disponibles dans chacune comme par exemple s' il dispose d'une station de gonflage, lavage, une boutique ou autre.

2. Identification des principales entités.

Nous avons donc identifié comme principales entités :

Station : avec un nom, des coordonnées, une adresse, une ville et une enseigne.

Carburant : avec un type de carburant.

Prix : avec une valeur et une date de mise à jour.

Rupture : avec une date de début, date de fin et une heure.

Horaire : avec une heure d'ouverture, et une heure de fermeture.

Services : avec un type de service fourni.

Stations Services : fait le lien entre un Station et ces Services.

3. Relation entre les tables.

Une station peut avoir plusieurs prix;

Mais un prix ne dépend que d'une station.

Station (0,n) —> Prix

Prix (1,1) —> Station

Un carburant peut avoir plusieurs prix.

Mais un prix ne dépend que d'un carburant.

Carburant (0,n) —> Prix

Prix (1,1) —> Carburant

Une station peut avoir plusieurs ruptures.

Mais une rupture ne concerne qu'une station. Rupture(1,1) —> Station

Station (0,n) —> Rupture

Un carburant peut avoir plusieurs ruptures.

Mais une rupture ne concerne qu'un carburant.

Carburant (0,n) → Rupture

Rupture (1,1) → Carburant

Une station peut avoir plusieurs horaires.

Mais un horaire ne dépend que d'une station.

Station (0,n) —> Horaire

Horaire (1,1) —> Station

Une station peut avoir plusieurs services.

Un service peut avoir plusieurs stations.

Station (0,n) —> Service

Service (0,n) —> Station

4. Clés primaires et clés étrangères.

L'entité Station possède une clé primaire : **id_station**.

L'entité Carburant possède une clé primaire : **id_carburant**.

L'entité Prix possède une clé primaire : **id_prix**,
et deux clés étrangères : *id_carburant*, qui fait référence à la table Carburant et *id_station* qui fait référence à la table Station

L'entité Rupture possède une clé primaire : **id_rupture**,
et deux clés étrangères : *id_carburant* qui fait référence à la table Carburant et *id_station* qui fait référence à la table Station

L'entité Horaire possède une clé primaire : **id_horaire**, et une clé étrangère *id_station* qui fait référence à la table Station.

L'entité Service possède une clé primaire : **id_service**.

L'Entité-association qui possède comme clé primaire : la concaténation des deux clés étrangères (*id_service* qui fait référence à la table Service, *id_station* qui fait référence à la table Station)

5. Schéma relationnel

Station (id_station, nom_station, latitude, longitude, adresse, ville, enseigne)

Carburant (id_carburant, type_carburant)

Prix (id_prix, *id_carburant*, *id_station*, val, date_maj)

Rupture (id_rupture, *id_station*, *id_carburant*, date_debut, date_fin)

Horaire (id_horaire, *id_station*, jour, heure_ouverture, heure_fermeture)

Service (id_service, type_service)

Station_Service(id_station, id_service)

La table **Horaire** permet de gérer les jours et heures d'ouverture d'une station. Ce choix va nous permettre ensuite d'identifier les stations ouvertes à un moment donné et de comparer uniquement les stations accessibles.

La table **Service** contient les différents services proposés par les stations, comme le lavage, la boutique ou la station de gonflage. Nous avons choisi de créer une table séparée afin d'éviter de répéter les mêmes services pour chaque station.

La table d'association **Station_service**, relie la table Station à la table Service, car une station peut proposer plusieurs services et un même service peut être présent dans plusieurs stations

La table d'association "**propose**" relie la table Station à la table Prix, car une station peut proposer plusieurs prix, car elle peut vendre plusieurs carburants, mais qu'un prix appartient à une seule station.

La table d'association "**réserve**" relie la table Carburant à la table Rupture, car un carburant peut être concerné par plusieurs ruptures, alors qu'une rupture concerne un seul carburant.

La table d'association "**subie**" relie la table Station à la table Rupture, car une station peut subir plusieurs ruptures de carburant, alors qu'une rupture concerne une seule station.

La table d'association "**est disponible**" relie la table Station à la table Horaire, car une station peut posséder plusieurs horaires d'ouverture, alors qu'un horaire appartient à une seule station.

La table d'association "**coûte**" relie la table Prix à la table Carburant, car un prix correspond à un seul carburant, alors qu'un carburant peut avoir plusieurs prix selon les stations et les dates de mise à jour.

II - Livrable 3

1. Scripts SQL de création des tables (Clés et contraintes) pour PSQL.

Nous avons dû ajouter une table qui fait le lien entre Horaire et Station que nous avons nommée Station_Horaire. Nous avons également rajouté le type de rupture à la table Rupture.

Ces tables vous nous permettent de contenir les données de nos 5 fichiers CSV modifiés à l'aide de python. L'ensemble du code qui va suivre est disponible dans le fichier "EnrichissementTable.sql" qui se situe dans le document : "Fichier/Script".

Station :

```
CREATE TABLE Station (  
    id_station INT PRIMARY KEY,  
    nom_station VARCHAR(100),  
    enseigne VARCHAR(100),  
    latitude REAL,  
    longitude REAL,  
    adresse VARCHAR(100),  
    ville VARCHAR(100)  
);
```

Carburant :

```
CREATE TABLE Carburant (  
    id_carburant INT PRIMARY KEY,  
    type_carburant VARCHAR(100)  
);
```

Prix :

```
CREATE TABLE Prix (  
    id_prix INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
```

```

        val REAL,
        date_maj DATE,
        id_carburant INT REFERENCES Carburant(id_carburant),
        id_station INT REFERENCES Station(id_station)
    );

```

Rupture :

```

CREATE TABLE Rupture (
    id_rupture INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY
KEY,
    date_debut DATE,
    date_fin DATE,
    type_rupture VARCHAR(50),
    id_station INT REFERENCES Station(id_station),
    id_carburant INT REFERENCES Carburant(id_carburant)
);

```

Horaire :

```

CREATE TABLE Horaire (
    id_jour INT PRIMARY KEY,
    jour VARCHAR(20),
    heure_ouverture REAL,
    heure_fermeture REAL
);

```

Station Horaire :

```

CREATE TABLE Station_Horaire (
    id_Jour INT REFERENCES Horaire(id_jour),
    id_Station INT REFERENCES Station(id_station),
    PRIMARY KEY(id_Jour, id_Station)
);

```

Service :

```

CREATE TABLE Service (
    id_service INT PRIMARY KEY,

```



```
        type_service VARCHAR(2000)
    );
```

Station Service :

```
CREATE TABLE Station_Service (
    id_station INT REFERENCES Station(id_station),
    id_service INT REFERENCES Service(id_service),
    PRIMARY KEY(id_station, id_service)
);
```

2. Scripts importation ou alimentation des données.

Pour alimenter notre base de données, nous avons utilisé le fichier CSV “**prix-des-carburants-en-france-flux-instantane-v2.csv**” provenant du site data.gouv.fr, elle contient les données des stations-service et des prix des carburants.

Pour le peuplement de la base, nous devons réaliser plusieurs étapes. Tout d’abord, nous devons corriger le fichier CSV à l’aide d’un script python.

En effet, ce programme lit chaque ligne du fichier CSV avec **csv.DictReader**, puis créer un nouveau fichier **prix-carburants.csv** en retirant les guillemets, en supprimant les espaces inutiles, présents dans le fichier original créant des erreurs avec la fonction **changer()**, en plus de cela, il remplace les virgules des nombres par des points pour faciliter les conversions en SQL avec la fonction **correction()**, qui est utilisée notamment pour les colonnes contenant des nombres comme les prix, la latitude ou la longitude. En effet, Ce script n’est pas notre première version, dans une des anciennes versions, nous avons utilisé la fonction **REPLACE** (par exemple 5,8210 devient 5.8210) car PostgreSQL n’interprète pas cela comme des virgules décimale, puis nous nous sommes dit que créer une fonction **correction()**, nous faciliterait la tâche (c’est pourquoi nous l’avons inclus directement dans le python). Le script récupère ensuite les informations importantes de chaque station, comme l’identifiant, les coordonnées, l’adresse, la ville, les prix des carburants, les dates de mise à jour, les ruptures, les services proposés ainsi que les horaires détaillés.

Il crée également un fichier **services.csv** chaque service apparaît une seule fois avec un identifiant unique, puis un fichier **station_services.csv** qui relie chaque station aux services qu'elle propose. Pour cela, nous avons utilisé un dictionnaire appelé **dico_services** qui va nous permettre d'associer chaque nom de service à un identifiant. Par exemple, si le service "Toilettes publiques" apparaît plusieurs fois dans le fichier CSV, il sera enregistré qu'une seule fois dans **services.csv**, avec le même identifiant à chaque fois. Lors de cette phase, nous avons eu des erreurs liées aux parenthèses et virgules, étant donné qu'une ligne est séparée par des virgules cela crée des erreurs avec "Vente de gaz domestique (Butane, Propane)", alors nous avons décidé de créer une fonction **Separer()** qui permet de découper correctement les services sans séparer les virgules qui sont entre parenthèses. Cette fonction parcourt le texte caractère par caractère et vérifie si la virgule se trouve à l'intérieur ou à l'extérieur des parenthèses. Si la virgule est dans les parenthèses, elle n'est pas utilisée pour couper le service.

De plus, le programme utilise également la liste **associations**, qui stocke les liens entre les stations et les services. Par exemple, si une station possède le service dont l'identifiant est 5, alors le fichier **station_services.csv** contiendra une ligne avec l'identifiant de la station et l'identifiant du service. Nous avons également utilisé une liste temporaire pour éviter d'ajouter deux fois le même service pour une même station.

Enfin, l'ajout sur les horaires va permettre de créer un fichier **horaires.csv** contenant les jours de la semaine avec leurs identifiants, ainsi qu'un fichier **station_horaires.csv** qui relie chaque station aux jours où elle est ouverte. Pour cela, nous avons utilisé le dictionnaire **dico_horaires**, qui associe chaque jour à un identifiant. Par exemple, Lundi correspond à l'identifiant 1, Mardi à l'identifiant 2, et ainsi de suite jusqu'à Dimanche. La fonction **Separer_Jour()** va ensuite nous permettre de vérifier quels jours sont présents dans la colonne des horaires détaillés donc si une station est ouverte le lundi, alors le fichier **station_horaires.csv** contiendra une ligne reliant l'identifiant de cette station à l'identifiant du lundi.

Mais nous avons ensuite décidé de modifier cette partie, car le fait de stocker seulement le jour n'était pas assez précis pour les requêtes SQL à faire par la suite. De ce fait, pour pouvoir savoir si une station est ouverte à une heure donnée, le jour d'ouverture à lui seul ne suffit pas, il faut également l'heure

d'ouverture et de fermeture. Nous avons donc modifié la création du fichier **horaires.csv** pour qu'il contienne maintenant ces quatre colonnes : **id_horaire**, **jour**, **heure_ouverture** et **heure_fermeture**. Par exemple, si une station est ouverte le lundi de 8h à 20h, alors le fichier horaires.csv contiendra comme cela : 1 ; Lundi ; 8 ; 20. Si l'heure contient des minutes, comme 20h30, alors elle est écrite sous la forme 20.30 pour faciliter son utilisation dans SQL, à l'aide de la fonction `Corriger_Heure()`, qui permet de transformer les horaires du fichier CVS dans un format plus simple. Par exemple, 08.00 devient 8, alors que 20.30 reste 20.30. Nous avons également ajouté une fonction `Recuperer_Horaires_Du_Jour()`, qui permet de récupérer les horaires présents dans la partie correspondant à un jour précis et la fonction `Separer_Horaire()`, qui permet de parcourir le texte des horaires détaillés et de retrouver pour chaque jour, les heures d'ouverture et de fermeture. Ce qui va nous permettre ensuite de créer un id différent pour chaque combinaison jour / heure d'ouverture / heure de fermeture. Nous avons aussi créé un dictionnaire `dico_horaires_id`, qui associe chaque horaire complet à un identifiant unique. Grâce à cela, le fichier horaires.csv ne contient pas plusieurs fois le même horaire. Si plusieurs stations ont exactement le même jour et les mêmes heures d'ouverture et de fermeture, elles utiliseront le même identifiant d'horaire.

Le fichier `station_horaires.csv` a aussi été modifié. Avant, il reliait une station à un simple jour, avec `id_station` et `id_jour`. Maintenant, il relie une station à un horaire précis, avec `id_station` et `id_horaire`.

Ce changement nous facilite vraiment la tâche dans les requêtes SQL, par exemple pour rechercher les stations ouvertes le lundi à 10h, ou pour comparer uniquement les prix des stations ouvertes à un moment précis. Les horaires deviennent plus exploitables que dans la dernière version du script, où nous conservons seulement les jours d'ouverture.

Ce traitement va nous faciliter ensuite l'alimentation des tables SQL, car les données sont déjà organisées en fichiers séparés correspondant aux différentes tables de la base : **stations**, **prix**, **services**, **station-services**, **horaires** et **station-horaires**.

Script Python pour les fichier csv :

Voir Annexes

Table temporaire :

```
DROP TABLE IF EXISTS Temporaire;  
DROP TABLE IF EXISTS TemporaireService;  
DROP TABLE IF EXISTS TemporaireHoraire;  
DROP TABLE IF EXISTS TemporaireStationService;  
DROP TABLE IF EXISTS TemporaireStationHoraire;
```

```
CREATE TABLE Temporaire (  
    id TEXT PRIMARY KEY,  
    latitude TEXT,  
    longitude TEXT,  
    code_postal TEXT,  
    pop TEXT,  
    adresse TEXT,  
    ville TEXT,  
    services TEXT,  
    prix TEXT,  
    rupture TEXT,  
    horaires TEXT,  
    geom TEXT,  
    prixGMJDATE TEXT,  
    prixG TEXT,  
    prixSP95MJDATE TEXT,  
    prixSP95 TEXT,  
    prixE85MJDATE TEXT,  
    prixE85 TEXT,  
    prixGPLcMJDATE TEXT,  
    prixGPLc TEXT,  
    prixE10MJDATE TEXT,  
    prixE10 TEXT,  
    prixSP98MJDATE TEXT,  
    prixSP98 TEXT,  
    ruptureE10 TEXT,  
    typeRuptureE10 TEXT,
```

```

    ruptureSP98 TEXT,
    typeRuptureSP98 TEXT,
    ruptureSP95 TEXT,
    typeRuptureSP95 TEXT,
    ruptureE85 TEXT,
    typeRuptureE85 TEXT,
    ruptureGPLc TEXT,
    typeRuptureGPLc TEXT,
    ruptureGazole TEXT,
    typeRuptureGazole TEXT,
    carburantDispo TEXT,
    carburantIndispo TEXT,
    carburantRuptureTemp TEXT,
    carburantRuptureDef TEXT,
    automate TEXT,
    servicesProposes TEXT,
    departement TEXT,
    codeDepartement TEXT,
    region TEXT,
    codeRegion TEXT,
    horaireDetails TEXT
);

CREATE TABLE TemporaireService (
    id INT PRIMARY KEY,
    type_serv TEXT
);

CREATE TABLE TemporaireStationService (
    id_Stat INT,
    id_Serv INT
);

CREATE TABLE TemporaireHoraire (
    id_jour INT,
    jour VARCHAR(20),
    horaire_ouverture REAL,
    horaire_fermeture REAL
);

CREATE TABLE TemporaireStationHoraire (
    id_Stat INT,
    id_jour INT
);

```

La table temporaire sert à recevoir toutes les données du fichier CSV. Elle contient les colonnes du fichier sous forme de texte afin de faciliter l'importation. Les conversions sont faites ensuite lors de l'insertion dans les tables finales.

```
\copy Temporaire FROM
/home/palugan/Downloads/prix-carburants.csv CSV HEADER
DELIMITER ';' ENCODING 'LATIN1';

\copy TemporaireService FROM
/home/palugan/Downloads/services.csv CSV HEADER DELIMITER ';'
ENCODING 'LATIN1';

\copy TemporaireStationService FROM
/home/palugan/Downloads/station_services.csv CSV HEADER
DELIMITER ';' ENCODING 'LATIN1';

\copy TemporaireStationHoraire FROM
/home/palugan/Downloads/station_horaires.csv CSV HEADER
DELIMITER ';' ENCODING 'LATIN1';

\copy TemporaireHoraire FROM
/home/palugan/Downloads/horaires.csv CSV HEADER DELIMITER ';'
ENCODING 'LATIN1';
```

Pour que l'import fonctionne correctement nous avons dû utiliser certaines options tel que "CSV HEADER", qui va indiquer que la première ligne du fichier correspond au noms des colonnes et ne doit donc pas être interprétée comme étant des données. Ensuite l'option "DELIMITER ';' ", qui va découper les lignes du fichiers selon le séparateur ";" car les fichiers fournis utilisent tous des points-virgules comme des séparateurs de champs.

Et finalement, l'option ENCODING 'LATIN1' est utilisée parce que le fichier contient des caractères accentués et qui ne sont pas encodés en UTF-8.

Enrichissement des tables:

Pour l'alimentation de la table Carburant étant donné que nous connaissons les types de carburants on peut la remplir manuellement. De plus, pour certaines colonnes nous n'avons pas assez de données pour les remplir, comme par exemple dans Station avec nom_station et enseigne, qui ne sont pas donnés dans les fichier CSV, ou encore date_debut et date_fin dans Rupture, qui ne sont pas données.

```
INSERT INTO Carburant (
VALUES
    (1, 'Gazole'),
    (2, 'SP95'),
    (3, 'E85'),
    (4, 'GPLc'),
    (5, 'E10'),
    (6, 'SP98')
);
```

```
INSERT INTO Station (
    id_station,
    latitude,
    longitude,
    adresse,
    ville
)
SELECT DISTINCT
    CAST(id AS INT),
    CAST(latitude AS REAL),
    CAST(longitude AS REAL),
    adresse,
    ville
FROM Temporaire;
```

```

INSERT INTO Prix (
    val,
    date_maj,
    id_carburant,
    id_station
)
SELECT
    CAST(prixG AS REAL),
    CAST(prixGMJDATE AS DATE),
    1,
    CAST(id AS INT)
FROM Temporaire;

INSERT INTO Prix (
    val,
    date_maj,
    id_carburant,
    id_station
)
SELECT
    CAST(prixSP95 AS REAL),
    CAST(prixSP95MJDATE AS DATE),
    2,
    CAST(id AS INT)
FROM Temporaire;

INSERT INTO Prix (
    val,
    date_maj,
    id_carburant,
    id_station
)
SELECT
    CAST(prixE85 AS REAL),
    CAST(prixE85MJDATE AS DATE),
    3,
    CAST(id AS INT)
FROM Temporaire;

```



```

INSERT INTO Prix (
    val,
    date_maj,
    id_carburant,
    id_station
)
SELECT
    CAST(prixGPLc AS REAL),
    CAST(prixGPLcMJDATE AS DATE),
    4,
    CAST(id AS INT)
FROM Temporaire;

```

```

INSERT INTO Prix (
    val,
    date_maj,
    id_carburant,
    id_station
)
SELECT
    CAST(prixE10 AS REAL),
    CAST(prixE10MJDATE AS DATE),
    5,
    CAST(id AS INT)
FROM Temporaire;

```

```

INSERT INTO Prix (
    val,
    date_maj,
    id_carburant,
    id_station
)
SELECT
    CAST(prixSP98 AS REAL),
    CAST(prixSP98MJDATE AS DATE),
    6,
    CAST(id AS INT)
FROM Temporaire;

```

```

INSERT INTO Rupture (
    type_rupture,
    id_station,
    id_carburant
)
SELECT
    CAST(typeRuptureGazole AS VARCHAR),
    CAST(id AS INT),
    1
FROM Temporaire;

```

```

INSERT INTO Rupture (
    type_rupture,
    id_station,
    id_carburant
)
SELECT
    CAST(typeRuptureSP95 AS VARCHAR),
    CAST(id AS INT),
    2
FROM Temporaire;

```

```

INSERT INTO Rupture (
    type_rupture,
    id_station,
    id_carburant
)
SELECT
    CAST(typeRuptureE85 AS VARCHAR),
    CAST(id AS INT),
    3
FROM Temporaire;

```

```

INSERT INTO Rupture (
    type_rupture,
    id_station,
    id_carburant
)
SELECT
    CAST(typeRuptureGPLc AS VARCHAR),
    CAST(id AS INT),
    4
FROM Temporaire;

```

```

INSERT INTO Rupture (
    type_rupture,
    id_station,
    id_carburant
)
SELECT
    CAST(typeRuptureE10 AS VARCHAR),
    CAST(id AS INT),
    5
FROM Temporaire;

```

```

INSERT INTO Rupture (
    type_rupture,
    id_station,
    id_carburant
)
SELECT
    CAST(typeRuptureSP98 AS VARCHAR),
    CAST(id AS INT),
    6
FROM Temporaire;

```

```

INSERT INTO horaire (
    id_jour,
    jour,
    heure_ouverture,
    heure_fermeture
)
SELECT DISTINCT
    id_jour,
    jour,
    horaire_ouverture,
    horaire_fermeture
FROM TemporaireHoraire;

```

```

INSERT INTO Station_Horaire (
    id_Jour,
    id_Station
)
SELECT DISTINCT
    id_jour,
    id_Stat
FROM TemporaireStationHoraire;

```

```

INSERT INTO Service (
    id_service,
    type_service
)
SELECT DISTINCT
    id,
    CAST(type_serv AS VARCHAR)
FROM TemporaireService;

```

```

INSERT INTO Station_Service (
    id_station,
    id_service
)
SELECT DISTINCT
    id_Stat,
    id_Serv
FROM TemporaireStationService;

```

3. Explication mécanisme d'historisation des prix.

Pour gérer l'historique des prix nous avons décidé de créer une table nommée Prix qui va contenir l'id d'un type de carburant, qui lui même est obtenue dans la table Carburant dans laquelle chaque type de carburant a un id. C'est donc une clé de référence vers la table Carburant.

Mais également un id faisant référence à une station qui se situe dans la table Station. Ce qui permet de savoir à quelle station appartient ce prix par exemple. En plus de ces quatre colonnes (id_prix, val (prix), id_carburant, id_station) nous avons ajouté à la table Prix, la colonne date_maj qui nous permet de savoir quand a été effectuée la dernière mise à jour sur le prix d'un type de carburant dans une station précise. Ce qui nous permet de savoir le prix le plus récent de chaque station.

Le fait d'ajouter de nouvelles lignes contenant, de nouvelles dates et donc de nouveaux prix ne change rien au résultat et nous permettra de voir l'évolution des prix, car cela ne supprimera pas l'ancienne au profit de la nouvelle, mais créera un nouvelle id_prix. Donc il y aura peut être de la redondance sur les informations comme le type de carburant et l'id de la station, cette méthode nous permet d'avoir les anciens prix et les nouveaux grâce au différentes versions du fichier CSV qui va nous permettre d'obtenir plus de données sur les prix au fur et à mesure que le fichier est mis à jour.

4. Ensemble des requêtes SQL d'analyse.

a. Analyse des stations et des carburants :

Afficher toutes les stations ayant des carburants disponibles :

```
SELECT DISTINCT id_station
FROM Prix
WHERE id_station IN (
    SELECT id_station
    FROM Rupture
    WHERE type_rupture <> 'definitive'
);
```

Explication :

On sélectionne de manière distincte (**SELECT DISTINCT**) les colonnes des identifiants des stations qui se retrouvent dans la table Prix (car il peut y avoir des doublons). Puis, dans la table qui y résulte, on y prend seulement les colonnes des identifiants qui se retrouve dans la sous-requête qui sélectionne les identifiants des stations dans la table Rupture qui n'ont pas comme type_rupture 'definitive'.

Afficher les prix associés à chaque carburant pour une station donnée :

```
SELECT s.id_station, s.nom_station, c.type_carburant, p.val AS
prix, p.date_maj
FROM Prix p
JOIN Station s ON p.id_station = s.id_station
JOIN Carburant c ON p.id_carburant = c.id_carburant
WHERE s.id_station = '42650001';
```

Explication :

On sélectionne les colonnes des identifiants des stations et leur nom (de la table Station), le type de carburant (de la table Carburant), la valeur du prix avec comme alias "prix" et enfin la date de mise à jour (de la table Prix). Pour relier ces tables entre eux, on va utiliser des jointures entre Prix et Station (avec comme lien l'identifiant de la station) et entre Prix et Carburant (avec comme lien l'identifiant du carburant). Pour finir, dans la table qui y résulte, on sélectionne seulement la colonne où l'identifiant a comme valeur '42650001'.

Identifier les stations ne proposant pas certains carburants :

```
SELECT r.id_station, s.nom_station, c.type_carburant
FROM Rupture r
JOIN Station s ON r.id_station = s.id_station
JOIN Carburant c ON r.id_carburant = c.id_carburant
WHERE type_rupture = 'temporaire' OR type_rupture =
'definitive';
```

Explication :

On sélectionne les colonnes des identifiants des stations (de la table Rupture), les noms des stations (de la table Station) et le type de carburant (de la table Carburant). Pour relier ces tables entre eux, on va utiliser des jointures entre Station et Rupture (avec comme lien l'identifiant de la station) et entre Rupture et Carburant (avec comme lien l'identifiant du carburant). Pour finir, dans la table qui y résulte, on sélectionne seulement les colonnes où le types de ruptures sont 'temporaire' ou 'definitive'.

Croiser les informations de stations, carburants et prix :

```
SELECT s.id_station, s.nom_station, s.ville,
c.type_carburant, p.val AS prix, p.date_maj
FROM Prix p
JOIN Station s ON p.id_station = s.id_station
JOIN Carburant c ON p.id_carburant = c.id_carburant;
```

Explication :

On sélectionne les colonnes des identifiants des stations, leur nom et leur ville (de la table Rupture), le type de carburant (de la table Carburant) et la valeur du prix avec comme alias 'prix' et leur date de mise à jour (de la table Prix). Pour relier ces tables entre eux, on va utiliser des jointures entre Station et Prix (avec comme lien l'identifiant de la station) et entre Prix et Carburant (avec comme lien l'identifiant du carburant).

b. Analyse des disponibilités

Identifier les stations en rupture pour un carburant donné :

```
SELECT r.id_station, s.nom_station
FROM Rupture r
JOIN Station s ON r.id_station = s.id_station
JOIN Carburant c ON r.id_carburant = c.id_carburant
WHERE c.type_carburant = 'E10' AND (type_rupture =
'temporaire' OR type_rupture = 'definitive');
```

Explication :

On sélectionne les colonnes des identifiants des stations (de la table Rupture) et les noms des stations (de la table Station). Pour relier ces tables entre eux, on va utiliser des jointures entre Station et Rupture (avec comme lien l'identifiant de la station) et entre Rupture et Carburant (avec comme lien l'identifiant du carburant). Cette dernière jointure nous permet d'avoir le type de carburant de la table Carburant. Enfin, de la table qui y résulte, on va prendre seulement la colonne où les carburants sont de type E10 et soit temporaire ou soit définitive

Analyser la fréquence des ruptures :

```
SELECT s.id_station
FROM Station s
JOIN Rupture r ON s.id_station = r.id_station
JOIN Carburant c ON r.id_carburant = c.id_carburant
GROUP BY s.id_station
HAVING COUNT(type_rupture = 'temporaire' OR type_rupture =
'definitive') > 4;
```

Explication :

On sélectionne les colonnes des identifiants des stations (de la table Station). Puis on va faire une jointure entre la table Station et Rupture (avec comme lien l'identifiant de la station) et une jointure entre Rupture et Carburant (avec comme lien l'identifiant du carburant). Rupture agit comme une “table de lien” liant Station et Carburant car il n'avait pas de clé en commun pour être lié. Puis, on va les regrouper grâce à un GROUP BY sur les identifiants des stations (de la table Station) afin d'utiliser le HAVING COUNT pour savoir les colonnes où le nombre de rupture, entre le type de rupture temporaire et définitive, est en dessous de 4

Croiser les informations de disponibilité avec les prix :

```
SELECT p.id_station, p.id_carburant, p.val
FROM Prix p
JOIN Rupture r ON p.id_station = r.id_station
WHERE p.val IS NOT NULL
GROUP BY p.id_station, p.id_carburant, p.val
HAVING COUNT(type_rupture = 'temporaire' OR type_rupture =
'definitive') < 4;
```

Explication :

On sélectionne les colonnes des identifiants des stations, les identifiants des carburants et la valeur du prix (de la table Prix). On a besoin d'une autre information qui est le type de rupture qui se retrouve dans la table Rupture. De ce fait, on va faire une jointure entre Rupture et Station (avec comme lien l'identifiant de la station). Après ça, on prend que les prix qui n'ont pas comme valeur 'null' (faut qu'il existe). On utilise un GROUP BY avec les identifiants des stations, les identifiants des carburants et la valeur du prix (de la table Prix) afin d'utiliser le HAVING COUNT pour savoir la colonne où le nombre de rupture entre le type de rupture temporaire et définitive. Puis on sélectionne seulement si la valeur est en dessous de 4

c. Analyse des horaires

Identifier les stations ouvertes à un moment donné :

```
SELECT DISTINCT s.id_station, s.nom_station, o.jour,
o.heure_ouverture, o.heure_fermeture
FROM Station s
JOIN Station_horaire h ON s.id_station = h.id_station
JOIN Horaire o ON h.id_jour = o.id_jour
WHERE o.jour = 'Dimanche' AND o.heure_ouverture = 9 AND
o.heure_fermeture = 20;
```

Explication :

On sélectionne de manière distincte (**SELECT DISTINCT**) les colonnes des identifiants des stations et leur nom (de la table Station) et le jour, l'heure d'ouverture et de fermeture (de la table Horaire). Pour relier ces tables, on utilise une jointure entre Station et Station_horaire (avec comme lien l'identifiant de la station). Ce lien est important car la table Station et Horaire ne sont pas liées. Le seul moyen de les relier est d'utiliser la table Station_horaire. De ce fait, on utilise une autre jointure entre Station_horaire et Horaire (avec comme lien l'identifiant du jour). De la table qui y résulte, on choisit seulement la colonne où le jour est 'Dimanche' et que l'heure d'ouverture est soit 9 (heure) ou soit 20 (heure).

Comparer les prix uniquement parmi les stations ouvertes :

```
SELECT s.id_station, s.nom_station,
MIN(p.val) AS prix_min, AVG(p.val) AS prix_moy,
MAX(p.val) AS prix_max
FROM Prix p
JOIN Station s ON p.id_station = s.id_station
JOIN Station_horaire h ON s.id_station = h.id_station
JOIN Horaire o ON h.id_jour = o.id_jour
WHERE o.jour = 'Samedi' AND p.val IS NOT NULL
GROUP BY s.id_station, s.nom_station;
```

Explication :

On sélectionne les colonnes des identifiants des stations et leur nom (de la table Station) et la valeur minimale de tous les prix avec comme alias 'prix_min', la moyenne de tous les prix avec comme alias 'prix_moy' et la valeur maximum de tous les prix avec comme alias 'prix_max' (de la table Prix). Pour relier ces tables, on utilise des jointures entre Prix et Station (avec comme lien l'identifiant de la station), entre Station et Station_horaire (avec comme lien l'identifiant de la station) et entre Station_horaire et Horaire (avec comme lien l'identifiant du jour). Puis on va chercher les colonnes où les jours sont 'Samedi' et où la valeur du prix n'est pas 'NULL'. Enfin, on les regroupe grâce au GROUP BY sur les identifiants des stations et leur nom (de la table Station).

Analyser les plages horaires d'ouverture des stations :

```
SELECT s.id_station, s.nom_station, COUNT(DISTINCT o.jour) AS
plage_horaires, ROUND(AVG(o.heure_ouverture)::numeric,2) AS
heure_ouverture_Moy, ROUND(AVG(o.heure_fermeture)::numeric,2)
AS heure_fermeture_Moy
FROM Station s
JOIN Station_horaire h ON s.id_station = h.id_station
JOIN Horaire o ON h.id_jour = o.id_jour
GROUP BY s.id_station, s.nom_station;
```

Explication :

On sélectionne les colonnes des identifiants des stations et leur nom (de la table Station), le nombre de jour distinctement (de la table Horaire) avec comme alias "plage_horaires", la moyenne d'ouverture (de la table Horaire) et la moyenne de fermeture (de la table Horaire). L'utilisation, sur 'heure_ouverture' et 'heure_fermeture', de la fonction AVG nous permet de faire concrètement la moyenne mais également l'utilisation d'un opérateur de typage '::numeric' nous permet de convertir le type de base (qui est de base un 'REAL') en un type supportable pour la fonction "ROUND()". De là, on a utilisé les jointures entre Station et Station_horaire (avec comme lien l'identifiant de la station) et entre Station_horaire et

Horaires (avec comme lien l'identifiant du jour) qu'on va regrouper dans un GROUP BY sur les identifiants des stations et leur nom (de la table Station)

d. Analyse comparative

Comparer les prix d'un même carburant entre différentes stations :

```
SELECT s.id_station, c.type_carburant, p.val
FROM Prix p
JOIN Station s ON s.id_station = p.id_station
JOIN Carburant c ON p.id_carburant = c.id_carburant
WHERE c.type_carburant = 'E10' AND p.val IS NOT NULL
GROUP BY s.id_station, c.type_carburant, p.val;
```

Explication :

On sélectionne les colonnes des identifiants des stations (de la table Station), le type de carburant (de la table Carburant) et la valeur du prix (de la table Prix). Pour relier ces tables entre eux, on va utiliser des jointures entre Prix et Station (avec comme lien l'identifiant de la station) et entre Prix et Carburant (avec comme lien l'identifiant du carburant). Pour finir, dans la table qui y résulte, on sélectionne seulement les colonnes où le type de carburant est 'E10' et la valeur du prix n'est pas 'NULL', puis on les regroupe grâce à un GROUP BY sur les identifiants des stations, le type de carburant et la valeur du prix.

Comparer les prix d'une station à différents moments :

Malheureusement, les fichiers CSV que nous disposons ne nous permettent pas d'avoir et de savoir les prix antérieurs d'une station, ce qui fait que nous ne pouvons pas réaliser cette requête.

Analyser les écarts de prix entre zones géographiques :

```
SELECT s.ville, AVG(p.val) AS prix_moy_ville,
MIN(p.val) AS prix_min_ville,
MAX(p.val) AS prix_max_ville,
(MAX(p.val) - MIN(p.val)) AS ecart_max
FROM Station s JOIN Prix p ON s.id_station = p.id_station
GROUP BY s.ville ;
```

Explication :

On sélectionne la colonne des villes (de la table Station), la moyenne de tous les prix avec comme alias 'prix_moy_ville', la valeur minimale de tous les prix avec comme alias 'prix_min_ville', la valeur maximum de tous les prix avec comme alias 'prix_max_ville' et la soustraction entre la valeur maximale et minimale de tous les prix avec comme alias 'ecart_max' (de la table Prix). Pour relier ces tables entre eux, on utilise une jointure entre Station et Prix (avec comme lien l'identifiant de la station). Enfin, on les regroupe grâce à un GROUP BY sur la colonne des villes (de la table Station).

e. Sélection basée sur des critères globaux

Identifier les stations dont les prix sont inférieurs à une moyenne :

```
SELECT p.id_station, AVG(p.val) AS moy_station
FROM Prix p
GROUP BY p.id_station
HAVING AVG(p.val) < (SELECT AVG(val) FROM Prix);
```

Explication :

On sélectionne la colonne des identifiants des stations et la moyenne de leurs prix avec comme alias 'moy_station' (de la table Prix). On va regrouper ces informations grâce à un GROUP BY sur les identifiants des stations (de la table Prix) afin d'utiliser le HAVING pour comparer la moyenne de chaque station. De là, on sélectionne seulement les colonnes où cette moyenne de station est strictement inférieure au résultat de la sous-requête qui calcule la moyenne globale de toutes les valeurs de prix de la table Prix.

Sélectionner les stations répondant à des critères combinés (prix, disponibilité, horaires) :

```
SELECT DISTINCT s.id_station
FROM Station s
JOIN Prix p ON s.id_station = p.id_station
JOIN Station_Horaire h ON p.id_station = h.id_station
JOIN Horaire o ON h.id_jour = o.id_jour
JOIN Rupture r ON r.id_station = s.id_station
WHERE p.val = 2 AND r.type_rupture IS NULL AND
(o.heure_ouverture = 8 OR o.heure_ouverture = 9) AND
o.heure_fermeture = 12;
```

Explication :

On sélectionne de manière distincte (SELECT DISTINCT) la colonne des identifiants des stations (de la table Station). Pour rassembler toutes les informations, on utilise des jointures successives entre Station et Prix (avec comme lien l'identifiant de la station), entre Prix et Station_Horaire (avec comme lien l'identifiant de la station), entre Station_Horaire et Horaire (avec comme lien l'identifiant du jour) et enfin entre Rupture et Station (avec comme lien l'identifiant de la station). Pour finir, de la table qui y résulte, on prend seulement les colonnes où la valeur du prix est égale à 2, où le type de rupture n'existe pas ('NULL'), où l'heure d'ouverture est soit 8 soit 9, et où l'heure de fermeture est égale à 12.

f. Analyse contextuelle**Déterminer les stations les plus avantageuses selon un contexte donné (zone, période, carburant) :**

```
SELECT s.id_station, s.nom_station, p.val AS prix
FROM Station s
JOIN Prix p ON s.id_station = p.id_station
JOIN Carburant c ON p.id_carburant = c.id_carburant
WHERE s.ville = 'Paris'
      AND c.type_carburant = 'SP95'
      AND p.date_maj >= '2025-05-01'
ORDER BY p.val;
```

Explication :

On sélectionne les colonnes des identifiants des stations et leur nom (de la table Station), et la valeur du prix avec comme alias "prix" (de la table Prix). Pour relier ces tables entre eux, on utilise des jointures entre Station et Prix (avec comme lien l'identifiant de la station) et entre Prix et Carburant (avec comme lien l'identifiant du carburant). De la table qui y résulte, on choisit seulement les colonnes où la ville est égale à 'Paris', le type de carburant est 'SP95' et la date de mise à jour est supérieure ou égale au '2025-05-01'. Enfin, on trie le résultat par ordre croissant selon la valeur du prix grâce au ORDER BY.

Comparer un enregistrement à un ensemble de valeurs dépendant de son contexte :

```
WITH MoyennesParVille AS (  
    SELECT s.ville, ROUND(AVG(p.val)::numeric,2) AS  
prix_moyen_ville  
    FROM Station s  
    JOIN Prix p ON s.id_station = p.id_station  
    GROUP BY s.ville  
)  
SELECT s.id_station, s.nom_station, s.ville, p.val AS  
prix_station, m.prix_moyen_ville, ROUND((p.val -  
m.prix_moyen_ville)::numeric,2) AS ecart_a_la_moyenne  
FROM Station s  
JOIN Prix p ON s.id_station = p.id_station  
JOIN MoyennesParVille m ON s.ville = m.ville;
```

Explication :

On crée d’abord une table temporaire nommée “MoyennesParVille” grâce au mot-clé WITH, dans laquelle on sélectionne la ville (de la table Station) et la moyenne des prix arrondie à deux décimales avec comme alias ‘prix_moyen_ville’ après une jointure entre Station et Prix et un regroupement par ville. Ensuite, dans la requête principale, on sélectionne l’identifiant, le nom et la ville (de la table Station), la valeur du prix avec comme alias ‘prix_station’ (de la table Prix), le prix moyen de la ville (de la table temporaire) et la soustraction arrondie entre le prix de la station et le prix moyen de la ville avec comme alias ‘ecart_a_la_moyenne’. Pour lier tout cela, on effectue une jointure entre Station et Prix (sur l’identifiant de la station) et une jointure entre Station et notre table temporaire (avec comme lien le nom de la ville).

g. Traitements complexes

Produire des analyses nécessitant plusieurs étapes de calcul :

```
WITH StationsFiables AS (  
    SELECT id_station  
    FROM Rupture  
    GROUP BY id_station  
    HAVING COUNT(type_rupture) < 2)  
SELECT s.ville, AVG(p.val) AS prix_moyen_stations_fiables  
FROM Prix p  
JOIN Station s ON p.id_station = s.id_station  
WHERE s.id_station IN (SELECT id_station FROM StationsFiables)  
GROUP BY s.ville;
```

Explication :

On crée d’abord une table temporaire nommée “StationsFiables” qui sélectionne et regroupe les identifiants des stations de la table Rupture qui ont un nombre total de ruptures strictement inférieur à 2. Dans la requête principale, on sélectionne la ville (de la table Station) et la moyenne des prix avec comme alias ‘prix_moyen_stations_fiables’ (de la table Prix) après avoir lié Prix et Station par l’identifiant de la station. Ensuite, dans la table qui y résulte, on utilise la clause WHERE pour ne garder que les colonnes dont l’identifiant de la station est présent dans notre table temporaire, puis on applique un GROUP BY sur les villes.

Structurer les traitements de manière lisible et évolutive :

```
SELECT DISTINCT s.id_station,
                AVG(o.heure_ouverture) AS horaire_ouverture_Moy,
                AVG(o.heure_fermeture) AS horaire_fermeture_Moy,
                AVG(p.val) AS prix_Moy
FROM Station s
JOIN Prix p ON p.id_station = s.id_station
JOIN Station_Horaire h ON h.id_station = s.id_station
JOIN Horaire o ON o.id_jour = h.id_jour
GROUP BY s.id_Station ;
```

Explication :

On sélectionne de manière distincte (SELECT DISTINCT) la colonne des identifiants des stations (de la table Station), la moyenne de l’heure d’ouverture avec comme alias ‘horaire_ouverture_Moy’, la moyenne de l’heure de fermeture avec comme alias ‘horaire_fermeture_Moy’ (de la table Horaire) et la moyenne des prix avec comme alias ‘prix_Moy’ (de la table Prix). Pour relier l’ensemble des données, on réalise des jointures entre Station et Prix (sur l’identifiant de la station), entre Station et Station_Horaire (sur l’identifiant de la station) et entre Station_Horaire et Horaire (avec comme lien l’identifiant du jour). Pour finir, on regroupe les lignes à l’aide d’un GROUP BY sur l’identifiant des stations (de la table Station).

h. Production d'indicateurs

Calculer des moyennes, minimums et maximums de prix :

```
SELECT c.type_carburant,  
       COUNT(p.id_prix) AS nombre_prix,  
       MIN(p.val) AS prix_min,  
       AVG(p.val) AS prix_moy,  
       MAX(p.val) AS prix_max  
FROM Prix p  
JOIN Carburant c ON p.id_carburant = c.id_carburant  
GROUP BY c.type_carburant;
```

Explication :

On sélectionne la colonne du type de carburant (de la table Carburant), le compte du nombre total d'identifiants de prix avec comme alias 'nombre_prix', la valeur minimale de prix avec comme alias 'prix_min', la moyenne de prix avec comme alias 'prix_moy' et la valeur maximale de prix avec comme alias 'prix_max' (de la table Prix). On utilise une jointure entre la table Prix et la table Carburant (avec comme lien l'identifiant du carburant) pour lier les informations. Enfin, on regroupe toutes ces lignes par type de carburant grâce à la clause GROUP BY.

Regrouper les données par carburant, station ou période :

```
SELECT p.id_station, c.type_carburant  
FROM Prix p  
JOIN Carburant c ON p.id_carburant = c.id_carburant  
GROUP BY p.id_station, c.type_carburant
```

Explication :

On sélectionne la colonne des identifiants des stations (de la table Prix) et le type de carburant (de la table Carburant). Pour relier les deux tables, on va utiliser une jointure entre la table Prix et la table Carburant (avec comme lien l'identifiant du carburant). Pour finir, de la table qui y résulte, on regroupe l'ensemble des lignes à l'aide d'un GROUP BY appliqué simultanément sur les identifiants des stations et sur le type de carburant.

Produire des indicateurs synthétiques exploitables :

```
SELECT s.id_station,  
       AVG(p.val) AS prix_moyen_enseigne,  
       COUNT(r.type_rupture) AS ruptures_actives  
FROM Station s  
LEFT JOIN Prix p ON s.id_station = p.id_station  
LEFT JOIN Rupture r ON s.id_station = r.id_station AND  
r.date_fin IS NULL  
GROUP BY s.id_station  
ORDER BY prix_moyen_enseigne ASC;
```

Explication :

On sélectionne la colonne des identifiants des stations (de la table Station), la moyenne de la valeur des prix avec comme alias 'prix_moyen_enseigne' (de la table Prix) et le compte du nombre de ruptures avec comme alias 'ruptures_actives' (de la table Rupture). Pour s'assurer de garder toutes les stations même celles qui n'ont pas de prix ou de ruptures, on va utiliser des jointures externes gauches (LEFT JOIN) entre Station et Prix d'une part, et entre Station et Rupture d'autre part (avec comme liens communs l'identifiant de la station) en ajoutant une condition pour ne compter que les ruptures sans date de fin. Pour finir, on regroupe par l'identifiant de la station avec un GROUP BY, puis on trie le résultat par ordre croissant selon le prix moyen obtenu.

5. Interprétation des premiers résultats.

Les premières requêtes SQL permettent d'analyser les stations-services selon les prix, les disponibilités des carburants et les horaires d'ouverture. Les résultats montrent que certaines stations proposent des prix plus avantageux que d'autres et que des ruptures de carburants peuvent apparaître régulièrement selon les zones et les types de carburants.

Les calculs de moyennes, minimums et maximums permettent également de comparer les prix entre plusieurs stations ou plusieurs villes? Grâce à ces analyses, l'entreprise peut observer les tendances du marché, comparer les différences de prix selon les zones géographiques et identifier les stations les plus fiables.

Les requêtes sur les horaires permettent aussi de repérer les stations ouvertes à certains moments afin de comparer uniquement les stations réellement accessibles aux utilisateurs. Cela permet à l'entreprise d'obtenir des analyses plus précises et exploitables.

Enfin, l'utilisation des jointures, regroupements, ... permet de produire des indicateurs synthétiques utiles pour faciliter l'analyse des données. Ces traitements aident l'entreprise à réaliser des études plus complètes sur les prix, les disponibilités des carburants et le fonctionnement général des stations-service.

II - Livrable 4

1. Scripts SQL de création des tables (Clés et contraintes) pour PSQL.

Vu que nous avons déjà réalisé la création du script SQL et la création des tables demandées lors du livrable 3 nous avons décidé de ne pas les remettre pour éviter toute forme de répétitions, vous pouvez cliquer sur le titre de cette partie pour être redirigé vers le grand 1 du livrable 3.

2. Scripts importation ou alimentation des données.

C'est de nouveau le cas que pour la réalisation du script d'importation ou d'alimentation des données qui a également déjà été demandée lors du livrable 3. Nous avons encore une fois décidé de ne pas les remettre pour éviter toute forme de répétitions, vous pouvez donc de nouveau cliquer sur le titre de cette partie pour être redirigé vers le grand 2 du livrable 3.

3. Ensemble des requêtes SQL d'analyse.

Pareil pour cette partie, nous avons encore fait le choix de ne pas les remettre pour éviter toute forme de répétitions, vous pouvez cliquer sur le titre de cette partie pour être redirigé vers le grand 4 du livrable 3, car l'ensemble des requêtes SQL d'analyse a été effectuée dans ce livrable.

Par ailleurs certaines des requêtes présentes dans le livrable 3 ont été utilisées dans Grafana, mais sont peut être modifiées pour mieux correspondre à certain type de visualisation notamment l'ajout de latitude et longitude dans certaines requêtes pour pouvoir réaliser des Geomap, ce qui rend certaine visualisation plus parlante.

4. Visualisations produites.

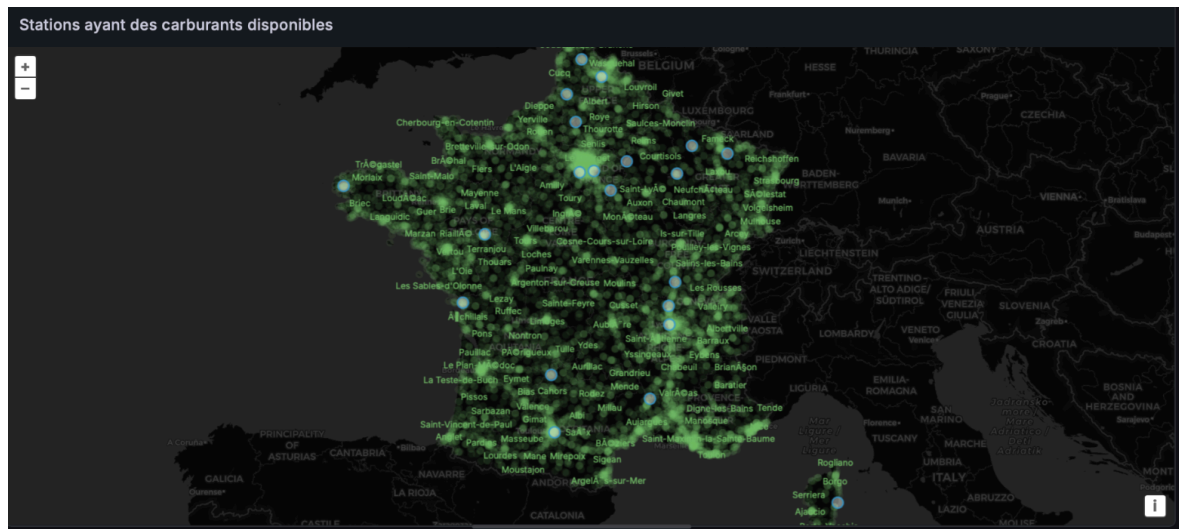
Lors de cette partie nous avons utilisé des requêtes du livrable 3 dans lesquelles certaines modifications ont pu être apportées. Pour comparer les différences entre les requêtes que nous avons prévues pour le terminal Linux et celles que nous avons prévues pour Grafana, le nom des requêtes sera mis en gras et souligné dès lors qu'elles ont été réalisées durant le livrable 3. Vous pouvez également trouver toutes ces visualisations dans le fichier json qui contient le dashboard Grafana.

Afficher toutes les stations ayant des carburants disponibles :

Requêtes dans Grafana :

```
SELECT DISTINCT
    s.id_station,
    s.ville,
    s.adresse,
    s.latitude / 100000.0 AS latitude,
    s.longitude / 100000.0 AS longitude
FROM Station s
JOIN Prix p ON s.id_station = p.id_station
WHERE s.latitude IS NOT NULL
    AND s.longitude IS NOT NULL
    AND p.val IS NOT NULL
    AND s.id_station IN (
        SELECT id_station
        FROM Rupture
        WHERE type_rupture <> 'definitive'
    );
```

Visualisation dans Grafana :



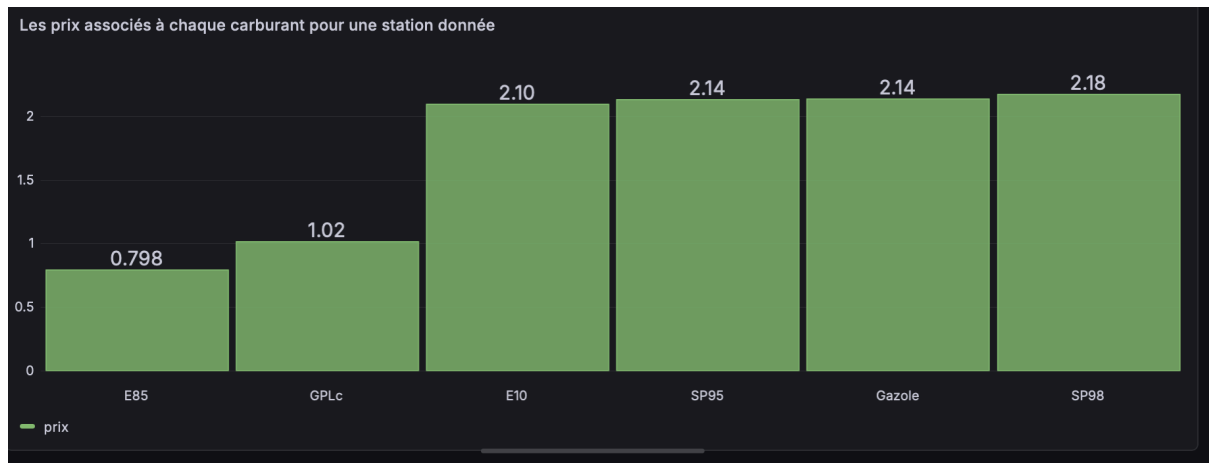
Nous avons décidé de réaliser cette requête avec un type de visualisation appelé Geomap, car nous trouvons que nous voyons mieux la répartition des différentes stations-services quand elle est visible sur une carte plutôt que dans un tableau avec des coordonnées. De plus, elle permet de voir les zones où il y a le plus de concentration de station-service avec la taille des points, car cette requête affiche les stations-services qui ont du carburant disponible.

Afficher les prix associés à chaque carburant pour une station donnée :

Requêtes dans Grafana :

```
SELECT
    c.type_carburant,
    p.val AS prix
FROM Prix p
JOIN Carburant c ON p.id_carburant = c.id_carburant
WHERE p.id_station = 3410001
ORDER BY p.val;
```

Visualisation dans Grafana :



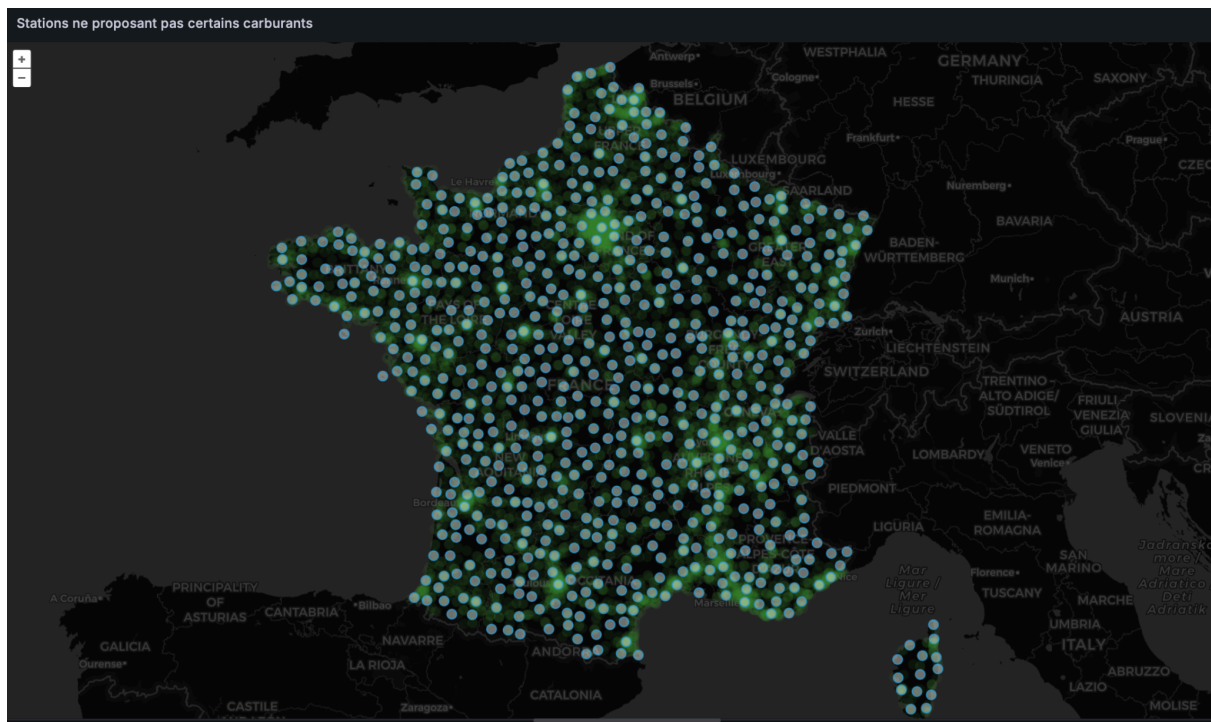
Dans cette requête nous avons décidé d'utiliser une visualisation de type bar chart, car nous avons trouvé que c'est la visualisation qui représente le mieux les écarts de prix des différents carburant de la station 3410001.

Identifier les stations ne proposant pas certains carburants :

Requêtes dans Grafana :

```
SELECT DISTINCT
  r.id_station,
  s.nom_station,
  s.ville,
  s.adresse,
  s.latitude / 100000.0 AS latitude,
  s.longitude / 100000.0 AS longitude,
  c.type_carburant,
  r.type_rupture
FROM Rupture r
JOIN Station s ON r.id_station = s.id_station
JOIN Carburant c ON r.id_carburant = c.id_carburant
WHERE type_rupture = 'temporaire' OR type_rupture =
'definitive'
AND s.latitude IS NOT NULL
AND s.longitude IS NOT NULL;
```

Visualisation dans Grafana :



Le but cette requête est donc de nous permettre d'identifier les stations qui ne propose pas un certain type de carburant, c'est à dire que dès lors qu'un carburant a un type de rupture 'temporaire' ou 'définitive' il va créer un point sur la Geomap, que nous avons encore utilisé car nous trouvons qu'une carte est plus judicieuse dans ce cas.

Croiser les informations de stations, carburants et prix :

Requêtes dans Grafana :

```
SELECT DISTINCT
  r.id_station,
  s.nom_station,
  s.ville,
  c.type_carburant,
  p.val AS prix,
  p.date_maj
FROM Prix p
JOIN Station s ON p.id_station = s.id_station
JOIN Carburant c ON p.id_carburant = c.id_carburant;
```

Visualisation dans Grafana :

Croisement entre stations, carburants et prix					
id_station	nom_station	ville	type_carburant	prix	date_maj
42700007		Firminy	Gazole	2.23	2026-05-19 02:00:00
30133001		Les Angles	Gazole	2.12	2026-05-19 02:00:00
11300006		Limoux	Gazole	2.13	2026-05-15 02:00:00
51390001		Vrigny	Gazole	2.46	2026-05-19 02:00:00
26400007		Crest	Gazole	2.24	2026-05-19 02:00:00
20620004		Biguglia	Gazole	2.25	2026-04-12 02:00:00
26800008		Portes-l'À's-Valence	Gazole	2.23	2026-05-18 02:00:00
34550001		Bessan	Gazole	2.21	2026-05-15 02:00:00
39700012		Rocheport-sur-Nenon	Gazole	2.15	2026-05-19 02:00:00

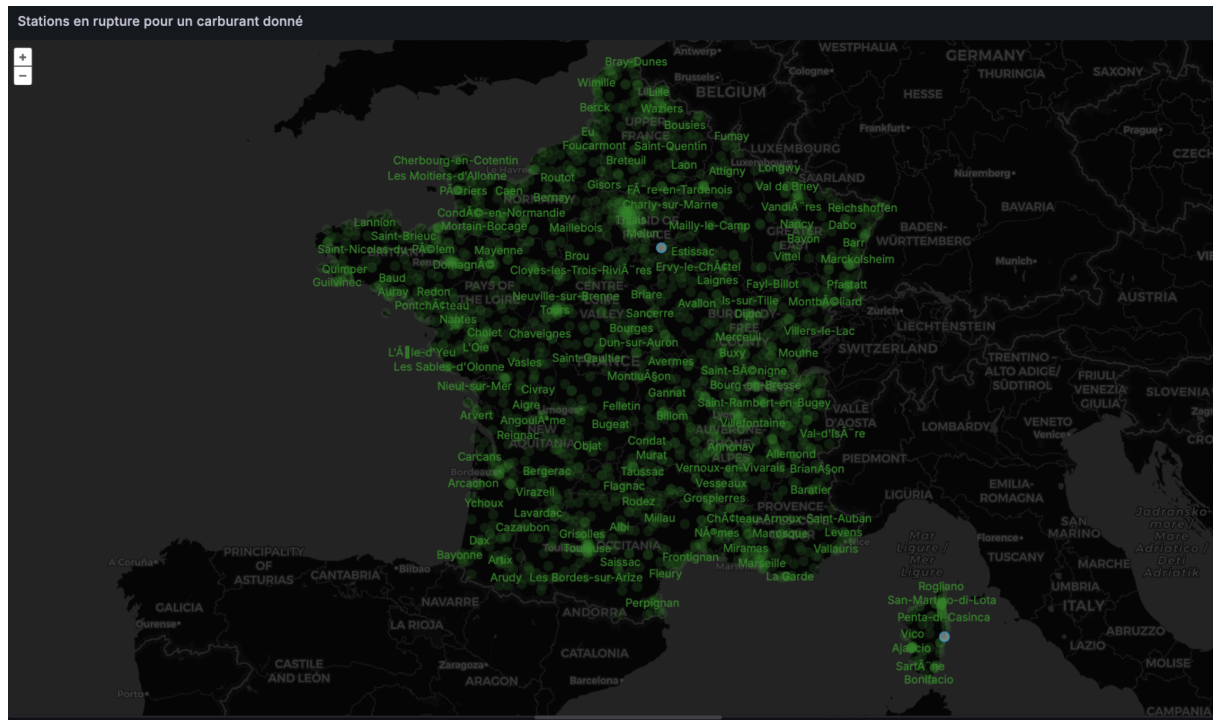
Pour cette requête nous avons trouvé que le meilleur choix de visualisation était une de type table, car faire un pie ou bar chart n'aurait pas été cohérent dans ce cas, mais c'est également le cas pour Geomap qui n'aurait pas permis de correctement comparer les prix entre les stations sur un carburant donnée.

Identifier les stations en rupture pour un carburant donné :

Requêtes dans Grafana :

```
SELECT DISTINCT
  r.id_station,
  s.nom_station,
  s.ville,
  s.adresse,
  s.latitude / 100000.0 AS latitude,
  s.longitude / 100000.0 AS longitude,
  c.type_carburant,
  r.type_rupture
FROM Rupture r
JOIN Station s ON r.id_station = s.id_station
JOIN Carburant c ON r.id_carburant = c.id_carburant
WHERE c.type_carburant = 'E10' AND (type_rupture =
'temporaire' OR type_rupture = 'definitive')
AND s.latitude IS NOT NULL
AND s.longitude IS NOT NULL;
```


Visualisation dans Grafana :



Pour mieux identifier les stations en rupture de E10 nous avons trouvé que la visualisation Geomap était encore une fois la solution la plus pratique et la plus facile pour analyser cette requête pour bien mettre en évidence des stations en rupture de E10.

Analyser la fréquence des ruptures :

Requêtes dans Grafana :

SELECT

```
s.id_station,  
s.ville,  
s.adresse,  
s.latitude / 100000.0 AS latitude,  
s.longitude / 100000.0 AS longitude,  
COUNT(r.type_rupture) AS nombre_ruptures
```

FROM Station s

JOIN Rupture r ON s.id_station = r.id_station

WHERE r.type_rupture IN ('temporaire', 'definitive')

AND s.latitude IS NOT NULL

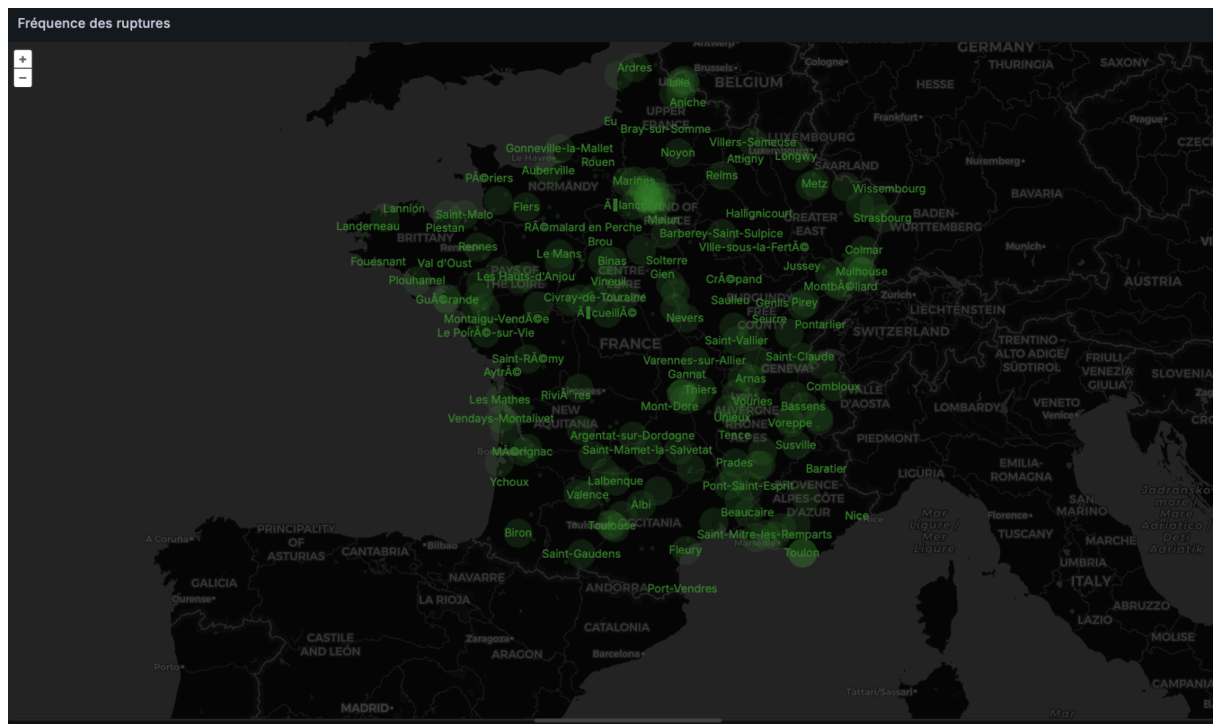
AND s.longitude IS NOT NULL

```

GROUP BY s.id_station, s.ville, s.adresse, s.latitude,
s.longitude
HAVING COUNT(r.type_rupture) > 4
ORDER BY nombre_ruptures DESC;

```

Visualisation dans Grafana :



Encore une fois nous avons trouvé qu'une visualisation Geomap était plus pertinente, car plus les points sont gros plus il y a de rupture ce qui permet de voir les zones les plus concernées par des ruptures de carburant très rapidement.

Croiser les informations de disponibilité avec les prix :

Requêtes dans Grafana :

```

SELECT p.id_station, p.id_carburant, p.val
FROM Prix p
JOIN Rupture r ON p.id_station = r.id_station
WHERE p.val IS NOT NULL
GROUP BY p.id_station, p.id_carburant, p.val
HAVING COUNT(type_rupture = 'temporaire' OR type_rupture =
'definitive') < 4 ORDER BY p.val ASC;

```

Visualisation dans Grafana :

Croisement des disponibilité avec les prix			
	id_station	id_carburant	val
	44119004	3	0.699
	44802001	3	0.699
	44522002	3	0.699
	29228001	3	0.699
	29480001	3	0.702
	17310001	3	0.705
	17310004	3	0.705
	47260001	3	0.706
	61100005	3	0.708

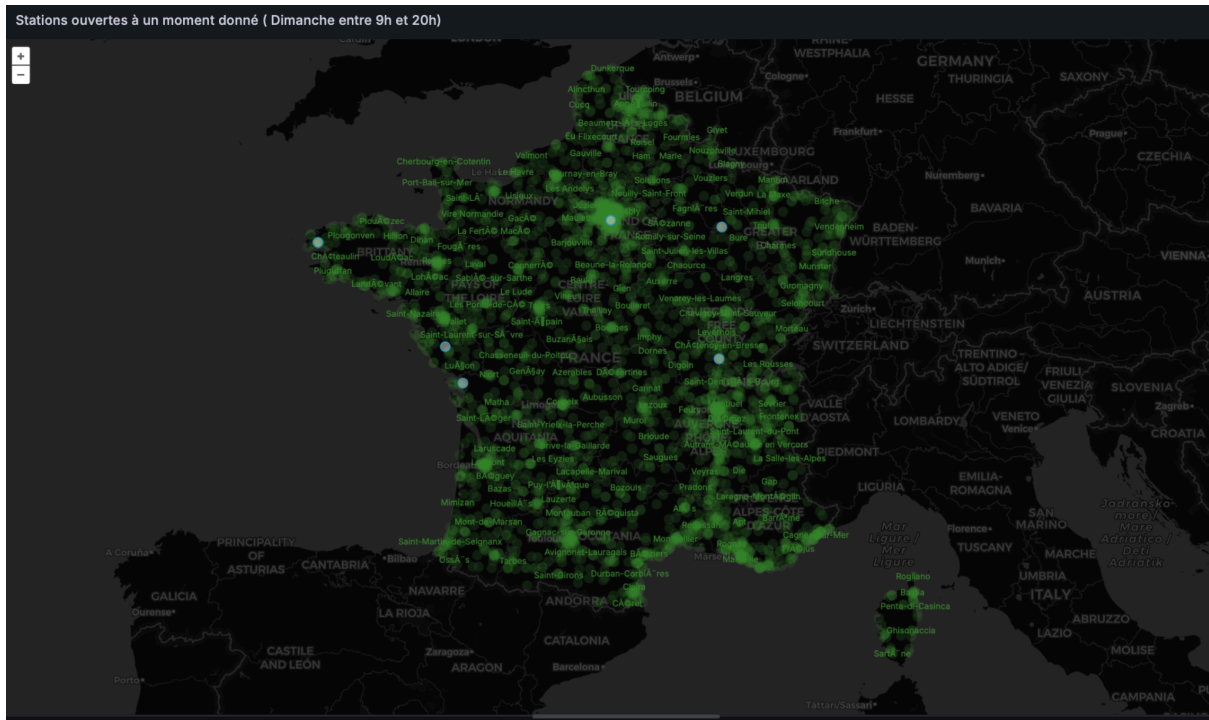
Lors de la création de cette visualisation nous avons trouvé que le plus judicieux pour cette requête qui nous permet d'obtenir les prix des carburant disponible pour chaque station était une visualisation de type table, car pour les autres visualisation nous aurions perdu de l'information notamment sur le prix des carburants.

Identifier les stations ouvertes à un moment donné :

Requêtes dans Grafana :

```
SELECT DISTINCT
  s.id_station,
  s.nom_station,
  s.ville,
  s.adresse,
  s.latitude / 100000.0 AS latitude,
  s.longitude / 100000.0 AS longitude,
  o.jour,
  o.heure_ouverture,
  o.heure_fermeture
FROM Station s
JOIN Station_horaire h ON s.id_station = h.id_station
JOIN Horaire o ON h.id_jour = o.id_jour
WHERE o.jour = 'Dimanche'
  AND o.heure_ouverture <= 9
  AND o.heure_fermeture >= 20
  AND s.latitude IS NOT NULL
  AND s.longitude IS NOT NULL;
```

Visualisation dans Grafana :



Pour cette visualisation nous avons utilisé une Geomap, car encore une fois cela nous permet d'obtenir de manière instantané, une bonne idée des différentes stations qui sont ouvertes de 9h à 20h le dimanche.

Comparaison des prix parmi les stations :

Requêtes dans Grafana :

```
SELECT
    id_station,
    MIN(val) AS prix_min
    ROUND(AVG(val)::numeric, 2) AS prix_moy,
    MAX(val) AS prix_max
FROM Prix
GROUP BY id_station;
```

Visualisation dans Grafana :

Comparer les prix parmi les stations ouvertes			
id_station	prix_min	prix_moy	prix_max
13124004	0.924	1.74	2.25
71370007	0.843	1.83	2.21
14440001	1.99	2.11	2.23
18150001	2.14	2.20	2.26
64260005	2.02	2.08	2.14
76191001	0.749	1.68	2.12
34990001	0.819	1.69	2.24
84290001	2.12	2.14	2.18
33790001	2.00	2.09	2.15

Nous avons trouvé que pour comparer le prix des carburants une visualisation de type table était la plus adéquate. Elle nous permet de voir correctement les écarts de prix entre les différentes stations notamment avec le prix moyen qui permet de savoir si la station a en moyenne des prix élevés ou plutôt bas.

Comparer les prix uniquement parmi les stations ouvertes :

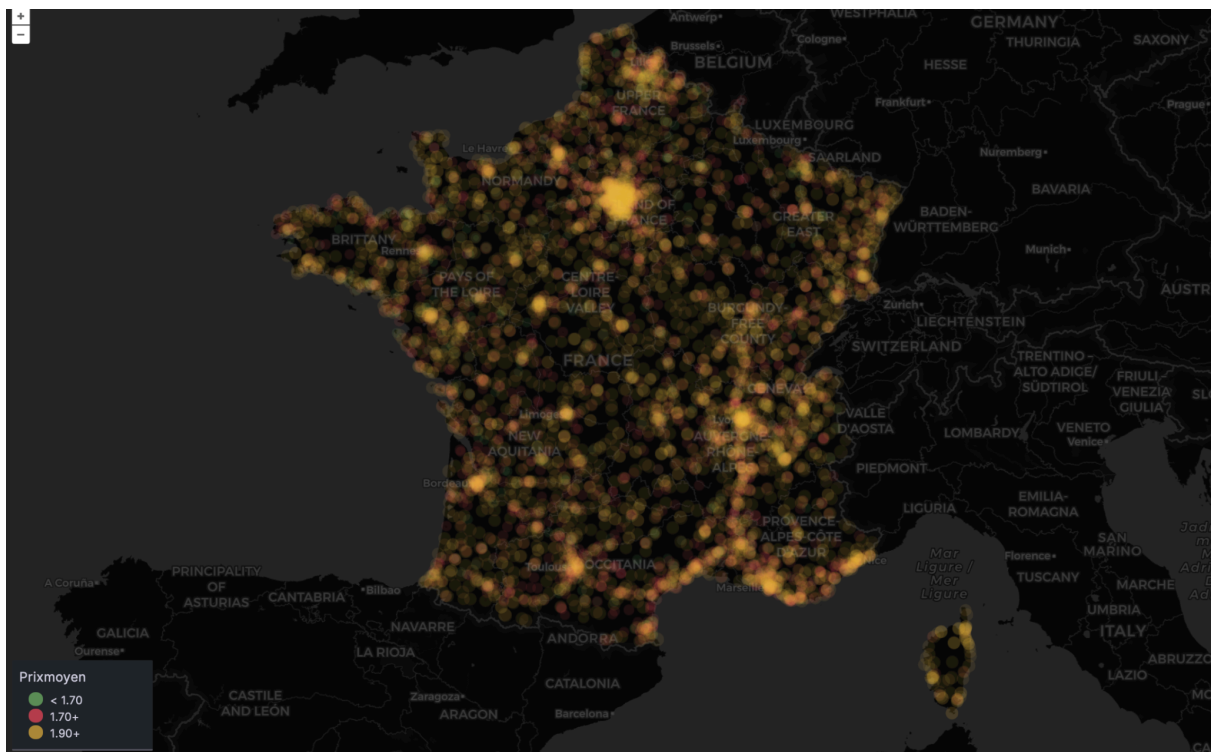
Requêtes dans Grafana :

```

SELECT
    s.id_station,
    s.nom_station,
    s.ville,
    s.adresse,
    s.latitude / 100000.0 AS latitude,
    s.longitude / 100000.0 AS longitude,
    AVG(p.val) AS prix_moy
FROM Prix p
JOIN Station s ON p.id_station = s.id_station
JOIN Station_horaire h ON s.id_station = h.id_station
JOIN Horaire o ON h.id_jour = o.id_jour
WHERE o.jour = 'Samedi'
    AND p.val IS NOT NULL
    AND s.latitude IS NOT NULL
    AND s.longitude IS NOT NULL
GROUP BY s.id_station, s.nom_station, s.ville, s.adresse,
s.latitude, s.longitude
ORDER BY prix_moy ASC;

```

Visualisation dans Grafana :



Cette requête nous montre presque la même chose que celle au dessus mais avec des nuances de couleur qui certes rendent moins précis les prix car nous n'avons pas exactement le prix moyen pour chaque station, mais nous permet de localiser les stations avec les prix les plus élevés.

Analyser les plages horaires d'ouverture des stations :

Requêtes dans Grafana :

```
SELECT s.id_station, s.nom_station, COUNT(DISTINCT o.jour) AS
plage_horaires, ROUND(AVG(o.heure_ouverture)::numeric,2) AS
heure_ouverture_Moy, ROUND(AVG(o.heure_fermeture)::numeric,2)
AS heure_fermeture_Moy
FROM Station s
JOIN Station_horaire h ON s.id_station = h.id_station
JOIN Horaire o ON h.id_jour = o.id_jour
GROUP BY s.id_station, s.nom_station;
```

Visualisation dans Grafana :

id_station	nom_station	plage_horaires	heure_ouverture_moy	heure_fermeture_moy
49160004		1	1	1
38114002		1	0	0
1750001		1	0	0
21120004		1	0	23.6
29630002		1	9	12
46300006		1	0	0
34070003		1	1	1
42130002		1	0	0
14130010		1	1	1

Cette requête nous permet de savoir le nombre de jours dans la semaine dans laquelle une station est ouverte mais également à quelle heure elle ouvre et ferme en moyenne. Nous avons décidé que la visualisation table était la meilleure des solutions car elle nous permet de correctement comparer les stations sur leurs horaires.

Comparer les prix d'un même carburant entre différentes stations :

Requêtes dans Grafana :

```
SELECT s.id_station, c.type_carburant, p.val
FROM Prix p
JOIN Station s ON s.id_station = p.id_station
JOIN Carburant c ON p.id_carburant = c.id_carburant
WHERE c.type_carburant = 'E10' AND p.val IS NOT NULL
GROUP BY s.id_station, c.type_carburant, p.val;
```

Visualisation dans Grafana :

id_station	type_carburant	val
1000001	E10	2.05
1000007	E10	2.07
1000009	E10	2.06
1000012	E10	1.99
1000013	E10	1.99
1000016	E10	1.99
1090001	E10	2.06
1100006	E10	2.04
1100007	E10	1.99

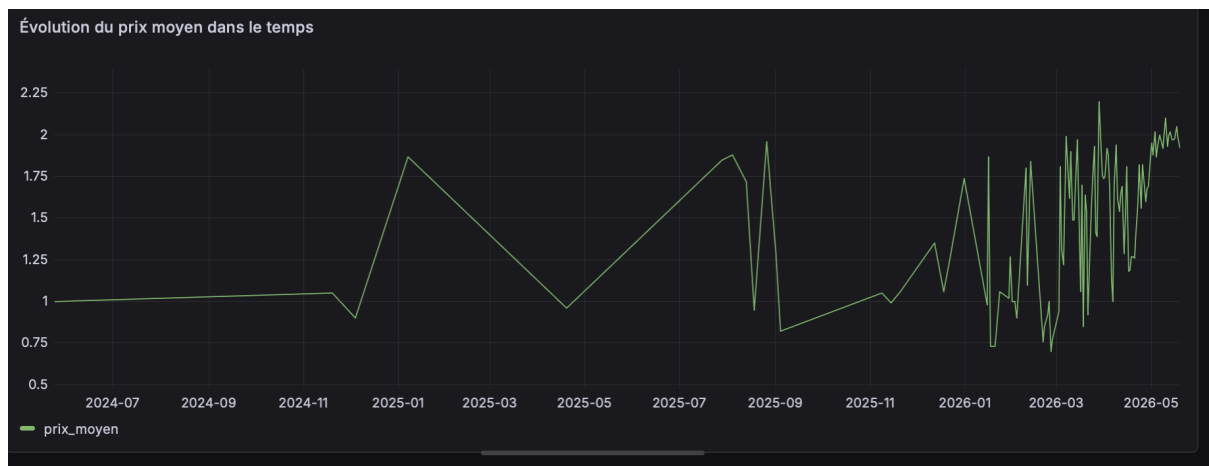
Une visualisation table a été choisie malgré une hésitation pour une Geomap car nous avons trouvé plus important de représenter cette donnée avec des prix concrets plutôt qu'une représentation sur une carte, c'est pourquoi nous affichons le prix du carburant E10 pour chaque station dans un tableau pour cette visualisation.

Évolution du prix moyen des carburants dans le temps :

Requêtes dans Grafana :

```
SELECT
    p.date_maj AS time,
    ROUND(AVG(p.val)::numeric, 2) AS prix_moyen
FROM Prix p
WHERE p.val IS NOT NULL
    AND p.date_maj IS NOT NULL
GROUP BY p.date_maj
ORDER BY p.date_maj;
```

Visualisation dans Grafana :



Pour cette requête la visualisation en time series était la plus appropriée car elle nous permet de voir correctement l'évolution du prix moyen des carburant dans le temps alors certe le début ne ressemble pas à une courbe mais cela ai du manque d'information. En revanche, vers 2026 les données montrent une réelle évolution du prix, c'est pourquoi, encore une fois, nous avons choisi cette visualisation.

Analyser les écarts de prix entre zones géographiques :

Requêtes dans Grafana :

```
SELECT s.ville, AVG(p.val) AS prix_moy_ville,  
MIN(p.val) AS prix_min_ville,  
MAX(p.val) AS prix_max_ville,  
(MAX(p.val) - MIN(p.val)) AS ecart_max  
FROM Station s JOIN Prix p ON s.id_station = p.id_station  
GROUP BY s.ville ;
```

Visualisation dans Grafana :

ville	prix_moy_ville	prix_min_ville	prix_max_ville	ecart_max
Brax	1.91	1.12	2.16	1.04
Cessenon-sur-Orb	2.14	2.10	2.17	0.0700
Gruissan	1.74	0.798	2.25	1.45
Iwuy	2.14	2.10	2.20	0.1000
Livers-Cazelles	1.99	1.99	1.99	0
Blet	2.12	1.99	2.25	0.260
La Rochefoucauld-en-Angour	2.01	0.758	2.30	1.54
La Roche-sur-Foron	2.11	1.99	2.22	0.230
	1.99	0.845	2.25	1.41

Nous avons choisi de représenter cette requête sous forme de visualisation table, car cela nous permet de correctement différencier les prix entre les différentes villes pour pouvoir la qu'elle a le meilleur prix moyen, le prix le plus bas, le prix le plus élevé ou encore l'écart entre ces prix le plus élevé.

Identifier les stations dont les prix sont inférieurs à une moyenne :

Requêtes dans Grafana :

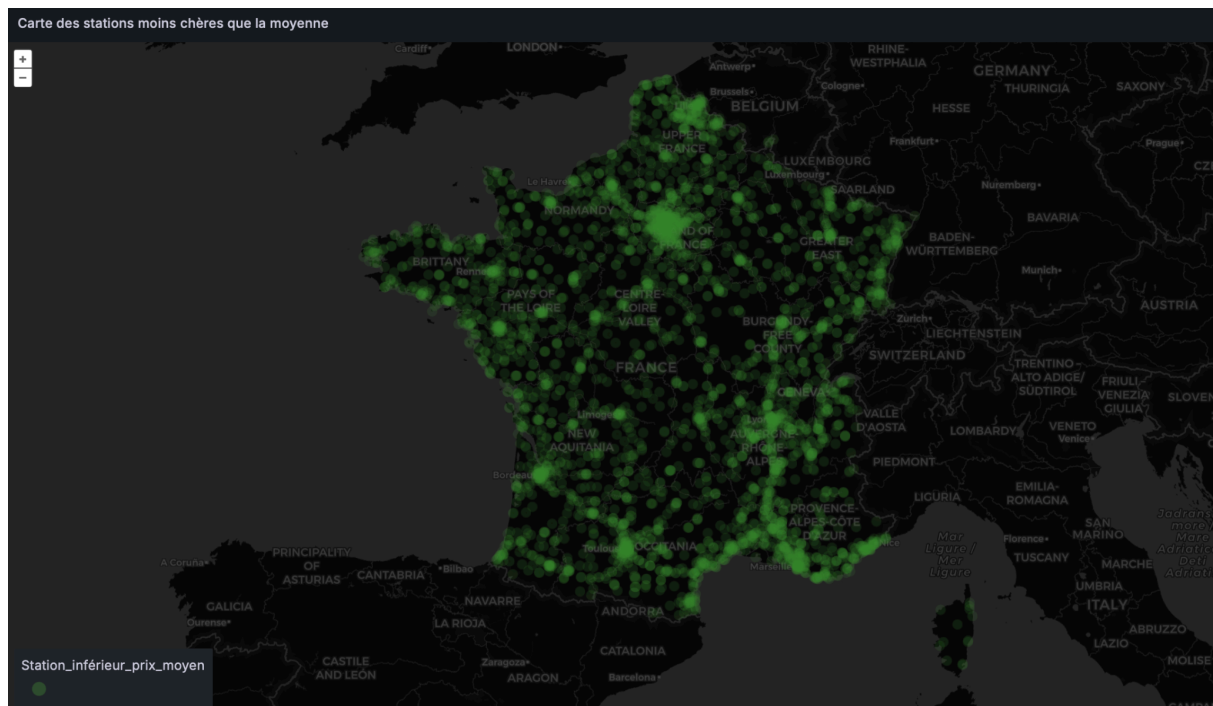
```
SELECT  
    s.id_station,  
    s.ville,  
    s.adresse,  
    s.latitude / 100000.0 AS latitude,
```

```

    s.longitude / 100000.0 AS longitude,
    AVG(p.val) AS moy_station
FROM Prix p
JOIN Station s ON p.id_station = s.id_station
WHERE p.val IS NOT NULL
    AND s.latitude IS NOT NULL
    AND s.longitude IS NOT NULL
GROUP BY s.id_station, s.ville, s.adresse, s.latitude,
s.longitude
HAVING AVG(p.val) < (SELECT AVG(val) FROM Prix WHERE val IS
NOT NULL)
ORDER BY moy_station ASC;

```

Visualisation dans Grafana :



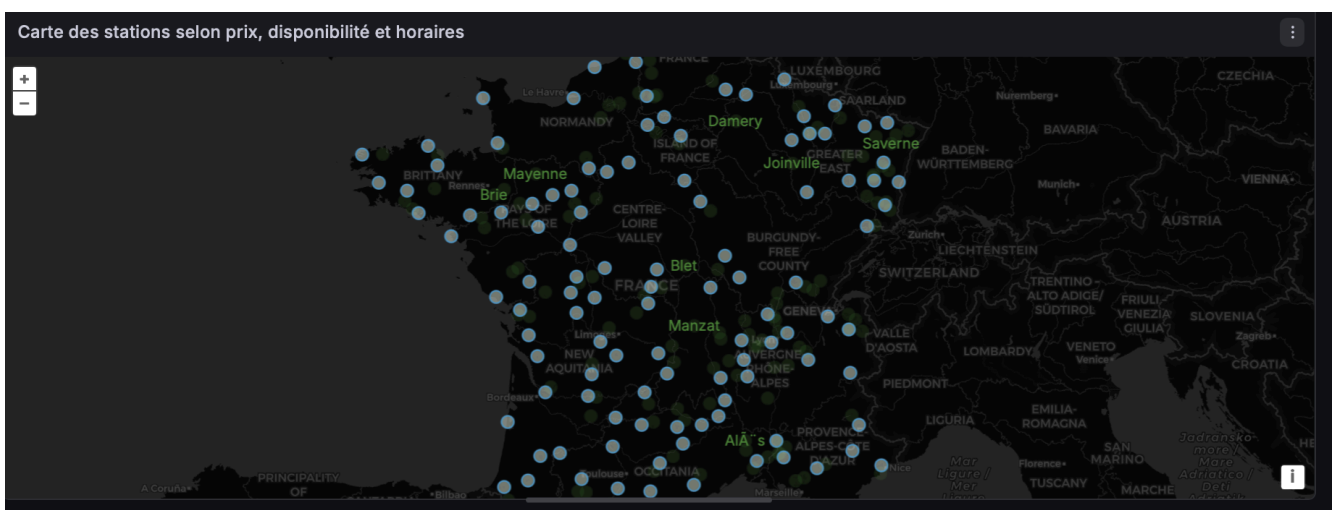
Cette carte met donc en avant les stations dont les prix sont inférieurs à la moyenne des prix nationaux, ce qui nous permet de les afficher sur une Geomap pour nous permettre de voir les stations où le carburant est souvent le moins cher que dans les autres stations.

Sélectionner les stations répondant à des critères combinés (prix, disponibilité, horaires) :

Requêtes dans Grafana :

```
SELECT DISTINCT
    s.id_station,
    s.ville,
    s.adresse,
    s.latitude / 100000.0 AS latitude,
    s.longitude / 100000.0 AS longitude,
    p.val AS prix,
    o.jour,
    o.heure_ouverture,
    o.heure_fermeture
FROM Station s
JOIN Prix p ON s.id_station = p.id_station
JOIN Station_Horaire h ON s.id_station = h.id_station
JOIN Horaire o ON h.id_jour = o.id_jour
LEFT JOIN Rupture r ON r.id_station = s.id_station
WHERE p.val <= 2
    AND (r.type_rupture IS NULL OR r.type_rupture <>
'definitive')
    AND (o.heure_ouverture = 8 OR o.heure_ouverture = 9)
    AND o.heure_fermeture = 12
    AND s.latitude IS NOT NULL
    AND s.longitude IS NOT NULL;
```

Visualisation dans Grafana :



Encore une fois nous avons choisi de représenter cette visualisation avec une Geomap, les point sur la carte représente les stations qui ont un carburant à 2 euros ou moins et qui sont ouvertes de 8h ou 9h jusqu'à 20h, sur ce cas là, nous trouvons qu'une carte était plus pertinente qu'un tableau.

Déterminer les stations les plus avantageuses selon un contexte donné (zone, période, carburant) :

Requêtes dans Grafana :

```
SELECT
    s.id_station,
    s.nom_station,
    s.ville,
    s.adresse,
    s.latitude / 100000.0 AS latitude,
    s.longitude / 100000.0 AS longitude,
    p.val AS prix
FROM Station s
JOIN Prix p ON s.id_station = p.id_station
JOIN Carburant c ON p.id_carburant = c.id_carburant
WHERE s.ville = 'Paris'
    AND c.type_carburant = 'SP95'
    AND p.date_maj >= '2025-05-01'
    AND p.val IS NOT NULL
    AND s.latitude IS NOT NULL
    AND s.longitude IS NOT NULL
ORDER BY p.val ASC;
```

Visualisation dans Grafana :



Nous avons fait une Geomap pour cette visualisation mais une table aurait aussi été adaptée. Nous avons dû zoomer car il n'y avait que 4 stations qui répondaient à notre requête, c'est pourquoi une table aurait très bien pu être utilisée dans ce cas.

Comparer un enregistrement à un ensemble de valeurs dépendant de son contexte :

Requêtes dans Grafana :

```
WITH MoyennesParVille AS (
    SELECT s.ville, ROUND(AVG(p.val)::numeric,2) AS
prix_moyen_ville
    FROM Station s
    JOIN Prix p ON s.id_station = p.id_station
    GROUP BY s.ville
)
SELECT s.id_station, s.nom_station, s.ville,p.val AS
prix_station, m.prix_moyen_ville, ROUND((p.val -
m.prix_moyen_ville)::numeric,2) AS ecart_a_la_moyenne
FROM Station s
JOIN Prix p ON s.id_station = p.id_station
JOIN MoyennesParVille m ON s.ville = m.ville;
ORDER BY ecart_a_la_moyenne ASC;
```

Visualisation dans Grafana :

id_station	nom_station	ville	prix_station	prix_moyen_ville	ecart_a_la_moyenne
18000021		Bourges	0.798	2.06	-1.26
18000022		Bourges	0.798	2.06	-1.26
75015036		Paris	0.825	2.08	-1.26
56700004		Hennebont	0.718	1.98	-1.26
30500003		Saint-Ambroix	0.798	2.06	-1.26
16110001		La Rochefoucauld-en-An	0.758	2.01	-1.25
40600004		Biscarrosse	0.739	1.99	-1.25
85150001		Les Achards	0.742	1.98	-1.24
87500004		Saint-Yrieix-la-Perche	0.798	2.04	-1.24

Cette requête nous permet de voir la différence de prix entre les stations d'une même ville comparer au prix moyen de celle-ci, c'est pourquoi nous avons choisi un tableau plutôt qu'une Geomap car nous aurions, certes les stations mais pas l'écart qu'il les séparer du prix moyen.

Produire des analyses nécessitant plusieurs étapes de calcul :

Requêtes dans Grafana :

```
WITH StationsFiabiles AS (  
    SELECT id_station  
    FROM Rupture  
    GROUP BY id_station  
    HAVING COUNT(type_rupture) < 2)  
SELECT s.ville, AVG(p.val) AS prix_moyen_stations_fiabiles  
FROM Prix p  
JOIN Station s ON p.id_station = s.id_station  
WHERE s.id_station IN (SELECT id_station FROM StationsFiabiles)  
GROUP BY s.ville;
```

Visualisation dans Grafana :

Prix moyen des stations fiables par ville	
ville	prix_moyen_stations_fiabiles
Le Grand-Bourg	1.73
Bagneux	2.22
Saint-Seurin-sur-l'Isle	1.65
Montval-sur-Loir	1.69
Marquise	1.76
Salernes	2.17
Rion-des-Landes	1.76
Le May-sur-Èvre	1.76
Villerbon	1.79

Vu qu'il nous fallait le prix des stations, nous avons opté pour une visualisation de type table, qu'un Geomap qui nous aurait juste afficher les stations. Cette requête nous permet donc de connaître la moyenne des prix des stations les plus fiables de chaque ville.

Structurer les traitements de manière lisible et évolutive :

Requêtes dans Grafana :

```
SELECT DISTINCT s.id_station,
    AVG(o.heure_ouverture) AS horaire_ouverture_Moy,
    AVG(o.heure_fermeture) AS horaire_fermeture_Moy,
    AVG(p.val) AS prix_Moy
FROM Station s
JOIN Prix p ON p.id_station = s.id_station
JOIN Station_Horaire h ON h.id_station = s.id_station
JOIN Horaire o ON o.id_jour = h.id_jour
GROUP BY s.id_Station ;
```

Visualisation dans Grafana :

id_station	horaire_ouverture_moy	horaire_fermeture_moy	prix_moy
25800001	0	23.6	1.79
71140001	0	23.6	2.20
59215001	0	23.6	2.15
54000014	7	22	2.14
38440003	1	1	1.79
31120003	7.37	20	2.24
26120004	0	23.6	1.84
49112001	1	1	2.09
64160001	0	23.6	1.80

Cette fois si la visualisation de type table est indispensable, car elle nous permet de savoir en détail la moyenne d'ouverture et de fermeture de la station ainsi que la moyenne des prix de ces carburants. C'est pourquoi la visualisation table est la meilleure dans ce cas.

Calculer des moyennes, minimums et maximums de prix :

Requêtes dans Grafana :

```
SELECT c.type_carburant,  
       COUNT(p.id_prix) AS nombre_prix,  
       MIN(p.val) AS prix_min,  
       AVG(p.val) AS prix_moy,  
       MAX(p.val) AS prix_max  
FROM Prix p  
JOIN Carburant c ON p.id_carburant = c.id_carburant  
GROUP BY c.type_carburant;
```

Visualisation dans Grafana :

Moyenne, minimum et maximum de prix par carburant				
type_carburant	nombre_prix	prix_min	prix_moy	prix_max
GPLc	9772	0.760	1.08	1.96
SP98	9772	1.83	2.14	2.70
Gazole	9772	1.81	2.17	2.89
E10	9772	1.88	2.05	2.50
SP95	9772	1.64	2.09	2.54
E85	9772	0.699	0.851	2.16

Encore une fois pour cette visualisation le type table était indispensable car aucune autre visualisation permettait d'afficher le nombre de prix associé à chaque carburants, mais également le fait d'afficher le prix minimum, maximum et moyen.

Regrouper les données par carburant, station ou période :

Requêtes dans Grafana :

```
SELECT p.id_station, c.type_carburant  
FROM Prix p  
JOIN Carburant c ON p.id_carburant = c.id_carburant  
GROUP BY p.id_station, c.type_carburant
```


Visualisation dans Grafana :

	id_station	type_carburant
	1000001	E10
	1000001	E85
	1000001	Gazole
	1000001	GPLc
	1000001	SP95
	1000001	SP98
	1000002	E10
	1000002	E85
	1000002	Gazole

Nous avons choisi de représenter cette requête avec une visualisation de type table, car c'est ce qui permet d'analyser le plus facilement cette requête qui nous permet de savoir qu'elle carburants disposent chaque stations.

Produire des indicateurs synthétiques exploitables :

Requêtes dans Grafana :

```
SELECT
    s.id_station,
    s.ville,
    ROUND(AVG(p.val)::numeric, 2) AS prix_moyen_station,
    COUNT(r.type_rupture) AS ruptures_actives
FROM Station s
LEFT JOIN Prix p ON s.id_station = p.id_station
LEFT JOIN Rupture r ON s.id_station = r.id_station
GROUP BY s.id_station, s.ville
ORDER BY prix_moyen_station ASC;
```

Visualisation dans Grafana :

id_station	ville	prix_moyen_station	ruptures_actives
58200008	Cosne-Cours-sur-Loire	0.800	30
28300008	Amilly	0.820	18
30250002	Aujargues	0.820	30
73200006	Albertville	0.830	30
88300008	Neufchâteau	0.840	30
67000027	Strasbourg	0.840	12
48000005	Mende	0.840	24
56440003	Languidic	0.840	18
34500025	Bâziers	0.850	30

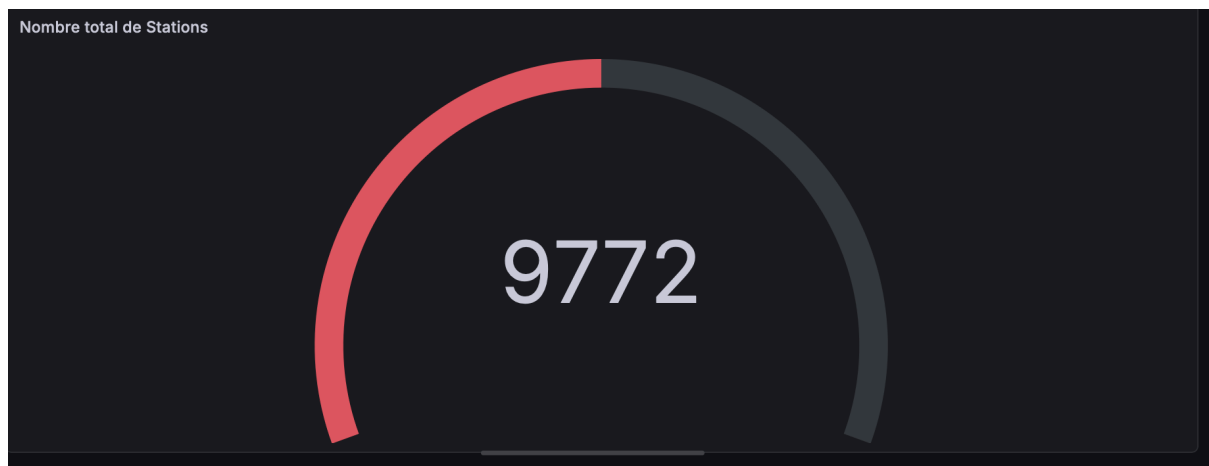
Nous avons pour une dernière fois utilisé une visualisation de type table qui nous a permis d'afficher les stations ainsi que la ville dans laquelle ils sont installés, mais le plus important et de savoir le prix moyen par station ainsi que de savoir le nombre de fois qu'une station a été en rupture.

Nombre total de stations :

Requêtes dans Grafana :

```
SELECT COUNT(*) AS total_stations
FROM Station;
```

Visualisation dans Grafana :



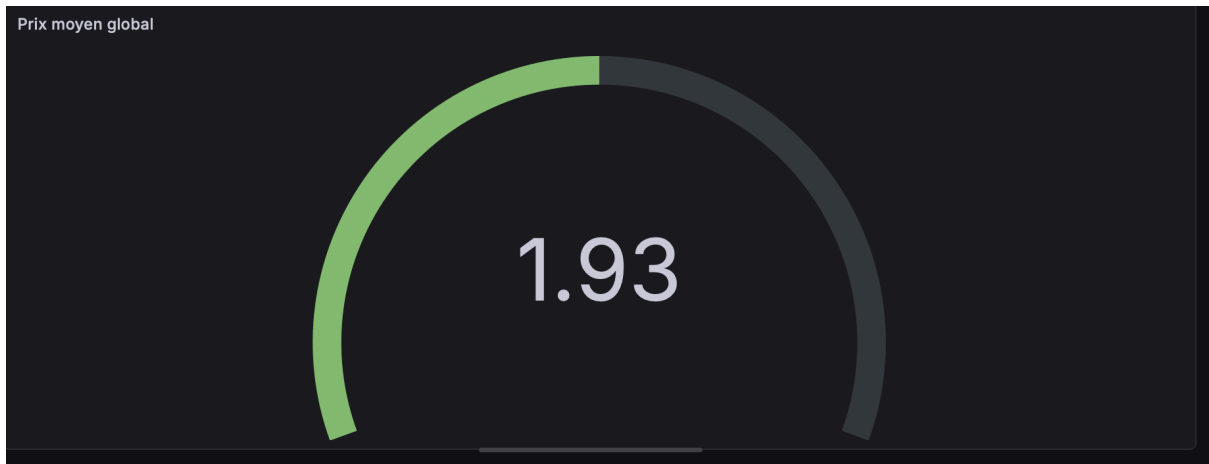
Cette visualisation de type stat nous permet de connaître en un clin d'œil le nombre total de stations dans la base de données.

Prix moyen global :

Requêtes dans Grafana :

```
SELECT AVG(val) AS prix_moyen_global
FROM Prix
WHERE val IS NOT NULL;
```

Visualisation dans Grafana :



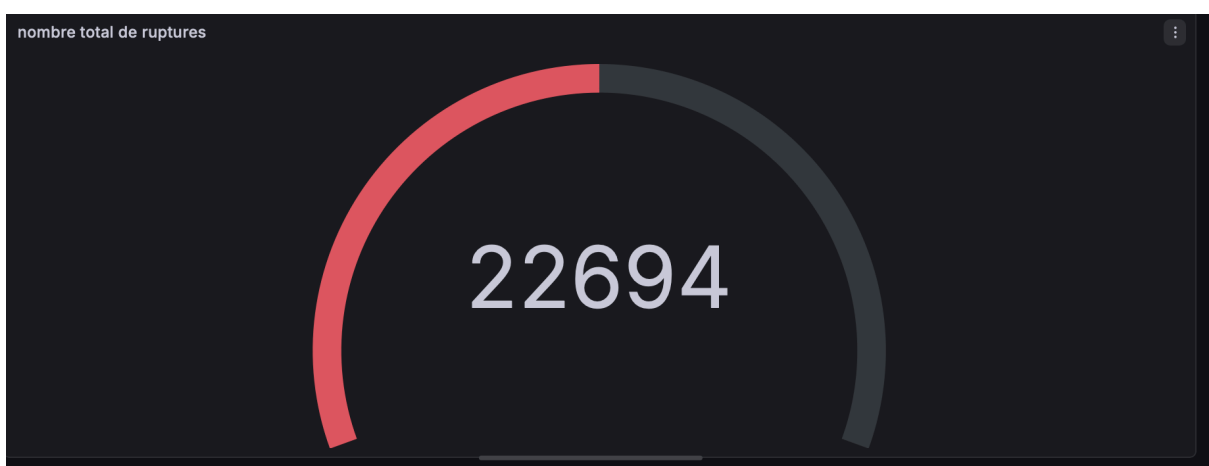
Cette visualisation de type stat nous permet de savoir le prix moyen global sur l'entièreté des prix et type de carburant.

Nombre total de ruptures:

Requêtes dans Grafana :

```
SELECT COUNT(*) AS total_ruptures
FROM Rupture
WHERE type_rupture IS NOT NULL AND type_rupture <> '';
```

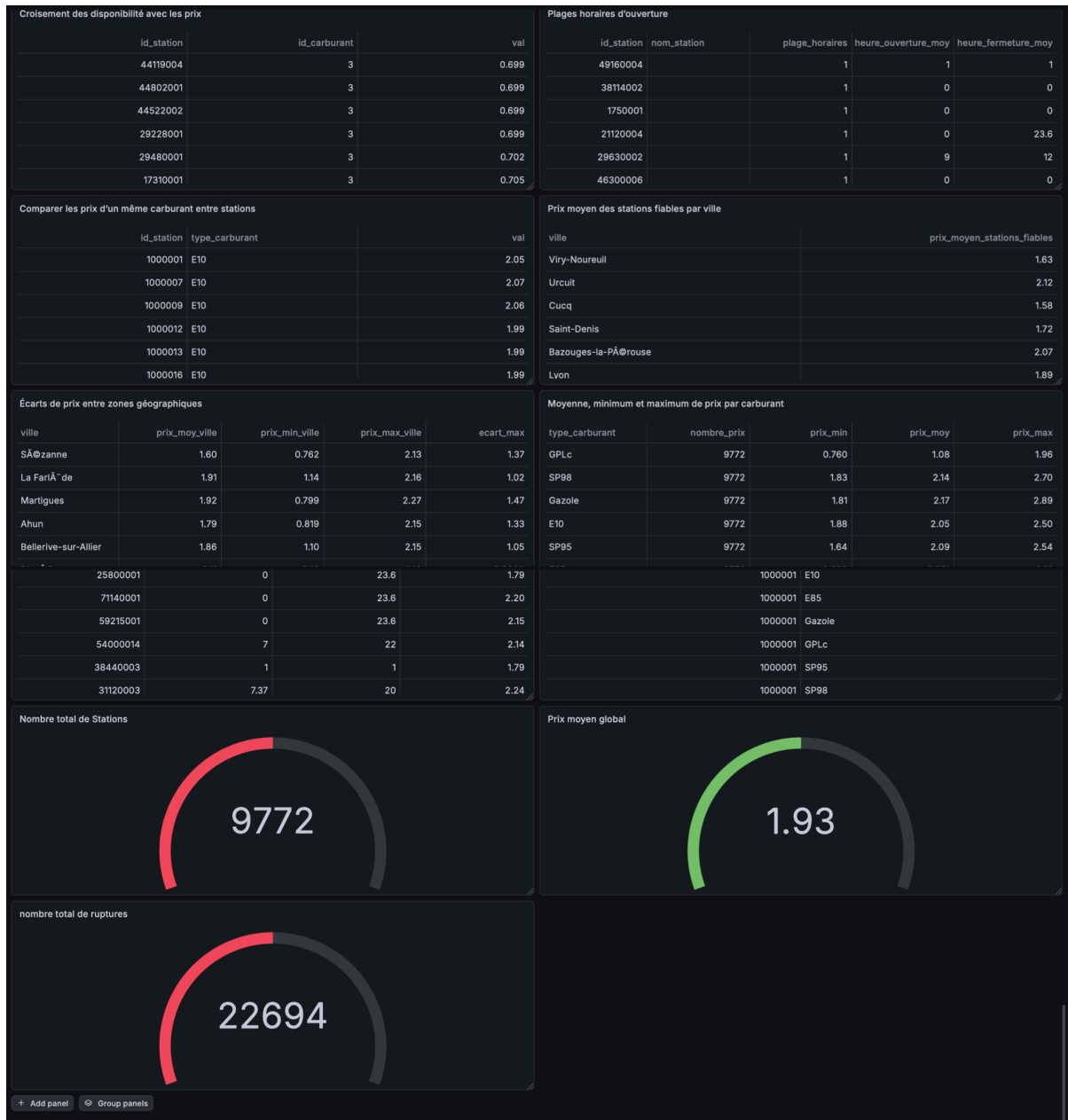
Visualisation dans Grafana :



Ce nombre représente le nombre total de rupture pour toutes les stations réunies, pour ce faire nous avons donc choisi une représentation de type stat.

5. Tableau de bord Grafana.





Pour la forme finale du Dashboard nous avons d  cid   de regrouper les diff  rentes visualisations en fonction de leur type dans un premier temps. C  est pourquoi les Geomap se trouvent en haut, les tables se situent vers le milieu juste apr  s le Time series et le Bar chart pour ensuite finir avec les stats. Mais nous les avons   galement tri  s en fonction de ce qu  elles nous permettent d  analyser c  est pour   a que les Geomap se trouvent en haut, car elles nous permettront d  identifier de fa  on g  ographique les stations alors que les tableaux vont plus nous informer sur les prix dans les stations. Nous avons   galement limit   les Pie et Bar chart car nous voulions utiliser le plus de donn  es possible sans utiliser de LIMIT.

6. Synthèse des résultats obtenus.

L'association des données du prix des carburants dans notre base de données PostgreSQL avec notre tableau de bord sous Grafana nous a permis d'éclairer plusieurs résultats majeurs et indicateurs clés des stations-service en France, qu'on a regroupées en 5 catégories.

Grâce aux visualisations de type *Geomap*, nous avons pu identifier que la base de données recense un total de **9 772 stations-service** sur le territoire français. La modélisation géographique (division par 100 000.0 des coordonnées initiales longitude et latitude) a rendu possible le placement précis de ces stations. Les cartes révèlent une forte **concentration** des infrastructures à proximité des grandes agglomérations (comme l'Île-de-France, la métropole lyonnaise ou la région marseillaise) et des axes routiers majeurs (les grands réseaux autoroutiers nationaux), facilitant le repérage visuel des points de vente actifs ou en rupture pour les utilisateurs.

L'analyse des indicateurs synthétiques globaux met en évidence un **prix moyen national de 1,93 €** tous carburants confondus. Cependant, le croisement des informations montre d'importantes disparités, c'est-à-dire des contrastes selon la nature du carburant et l'emplacement. On peut citer par exemple le **GPLc** et l'**E85** se positionnent comme les carburants les plus économiques (=rentables) avec des prix moyens respectifs de **1,08 €** et **0,851 €**, et des prix minimums débutant dès **0,699 €**. Tandis que le **Gazole**, l'**E10**, le **SP95** et le **SP98** affichent des moyennes bien plus élevées, fluctuant entre **2,05 €** pour le **E10** et **2,17 €** pour le **Gazole**, avec des pics maximaux atteignant jusqu'à **2,89 €** pour le gazole dans certaines stations, tel que celle de Vrigny (**51390001**) qui affiche un prix très élevé de **2,46 €** suivi de Firminy (**42700007**) avec un tarif de **2,23 €** ou encore de Crest (**26400007**) qui propose le Gazole à 2,24 €. (liste évidemment non exhaustive)

De plus, l'analyse comparative des écarts de prix entre zones géographiques (par ville) révèle de réelles inégalités locales. En effet, certaines communes affichent un écart maximal de prix presque nul entre leurs stations, comme **lwuy** ou **Livers-Cazelles** (avec de très faibles variations), tandis que d'autres, à l'image de **Gruissan** ou **La Rochefoucauld**, subissent des **écarts internes de plus de 1,40 €** pour un même carburant. La requête calculant l'écart à la moyenne montre que certaines stations isolées ou d'enseignes spécifiques proposent des tarifs allant jusqu'à **1,26 € en dessous de la moyenne de leur propre ville**.

Le suivi de l'évolution du prix moyen à travers les graphiques de temps de Grafana montre une instabilité et des fluctuations constantes sur la période s'étendant de 2024 à moitié-2026. Ces variations mettent en lumière l'importance d'un mécanisme d'historisation rigoureux (via notre table Prix), sans lequel l'entreprise n'aurait pas la possibilité d'identifier les périodes et les points de vente systématiquement les plus avantageux.

De même, le phénomène d'indisponibilité des carburants est un indicateur critique. Notre visualisation comptabilise un nombre massif de **22 694 ruptures** enregistrées (qu'elles soient **temporaires** ou **définitives**). Ce volume élevé s'explique par le fait que de nombreuses infrastructures subissent des ruptures à répétition ou sur plusieurs carburants simultanément, ce qui représente une moyenne de 2,3 ruptures par station sur l'ensemble du réseau. Lancées sur une Geomap avec des points proportionnels (plus le point est gros, plus le nombre de ruptures est élevé), ces données mettent en évidence les zones en tension. La carte révèle ainsi que si de nombreuses stations dépassent régulièrement les 4 ruptures distinctes, certains établissements cumulent jusqu'à 30 ruptures actives ou passées (comme à **Cosne-Cours-sur-Loire** ou **Aujargues**), devenant ainsi des points de vente jugés "peu fiables" pour l'approvisionnement des usagers.

L'exploitation des horaires détaillés à l'aide de scripts Python et de requêtes d'agrégation SQL a permis de filtrer finement les stations selon leur accessibilité réelle. Un grand nombre de stations fonctionnent sur le modèle *ouverture non-stop* (une ouverture de 0h à 23.6h, soit 23h36), offrant une flexibilité maximale. Le croisement des horaires et des prix permet d'affiner l'expérience de l'entreprise (/utilisateur) en ne comparant, par exemple, que les prix des stations effectivement ouvertes le samedi ou le dimanche dans un intervalle horaire précis (exemple : entre 9h et 20h), évitant ainsi de diriger un usager vers une station fermée ou en rupture complète.

En somme, les résultats obtenus démontrent l'efficacité de notre visualisation développée grâce à une fusion entre PostgreSQL et Grafana. En combinant la géolocalisation des **9 772 stations**, **l'historisation des prix** face aux fortes disparités du marché et le suivi des **22 694 ruptures**, le système apporte une vision complète et critique du réseau français. Associée à l'analyse précise des plages horaires d'ouverture, la base de données est capable de générer des indicateurs décisionnels. Elle permet ainsi à une entreprise (comme celle de la SAE) d'identifier instantanément les stations les plus fiables, ouvertes et avantageuses dans un contexte donné

7. Conclusion sur les principaux enseignements du projet.

Ce rapport, fait par notre équipe “Les phacochères pas trop chers”, nous a donné une opportunité de mobiliser et de consolider l'ensemble des compétences de conception, d'implémentation et d'exploitation d'une base de données relationnelle dans un contexte réaliste et professionnel, grâce au cours d'exploitation d'une base de donnée

Ce projet nous a également confronté à la complexité de la manipulation de données réelles et ouvertes (Open Data). Nous avons appris à, d'une part, nettoyer et structurer les données. En effet, en utilisant un script Python assez complexe mais précis on a pu faire un passage de données brutes à un modèle relationnel propre afin de segmenter les informations (extraction des prix, gestion des coordonnées géographiques et des états de rupture).

De plus, on a pu apprendre à exploiter la puissance du langage SQL grâce à PostgreSQL. En effet, en concevant des requêtes complexes sous PostgreSQL (calculs d'écarts par rapport à la moyenne de la ville, détection des anomalies de prix à moins de 0,75 € ou des historiques de ruptures), on a pu comprendre comment transformer des millions de lignes en indicateurs exploitables et utilisable, notamment pour une application de visualisation de données.

Grâce à Grafana, la mise en place des tableaux de bord dynamiques, nous a appris l'importance de transformer des données techniques en indicateurs d'aide à la décision, notamment grâce aux cartes Geomap permettant d'identifier immédiatement, par exemple les zones en tension et les stations les moins fiables.

La SAE nous a également appris à répondre précisément à un cahier des charges réaliste. Ce rapport permet à l'entreprise de posséder un outil complet d'analyse des stations-service, en intégrant simultanément quatre visions importantes tel que, les prix (mise en évidence d'écarts internes de plus de 1,40 € dans certaines villes), la disponibilité (comptabilisation des 22 694 ruptures pour anticiper les manques de carburants), la diversité des carburants (E85, GPLc, Gazole, SP98) et aussi les conditions d'accès (Filtrage intelligent par plages horaires).

En somme, ce projet nous a permis de démontrer notre capacité à mener un processus complet d'exploitation des données et de la visualisation, depuis leur extraction en passant par leur modélisation jusqu'à leur restitution sous forme d'indicateurs importants pour aider à la décision des utilisateurs. Cette SAE nous a sensibilisés aux enjeux liés à la distribution de l'énergie en France, tout en renforçant une approche rigoureuse et structurée, omniprésente dans le monde professionnel.

Annexes :

```
import csv

debut = "prix-des-carburants-en-france-flux-instantane-v2.csv"

fichier_prix= "prix-carburants.csv"
fichier_services = "services.csv"
fichier_station_services = "station_services.csv"
fichier_horaires = "horaires.csv"
fichier_station_horaires = "station_horaires.csv"

def changer(val):
    if val is None:
        return ""

    val = val.replace("'", "") #retire les guillemets du CSV
    val = val.strip()

    return val

def correction(val):
    val = changer(val)
    val = val.replace(",", ".") #change les virgules en points,
    pour les nombres flottants
    return val

def Separer(texte):
    services_L = []
    service = ""
    nbr_parenthese = 0

    for carac in texte:
        if carac == "(":
            nbr_parenthese = nbr_parenthese + 1
            service = service + carac

        elif carac == ")":
            nbr_parenthese = nbr_parenthese - 1
            service = service + carac
```

```

        elif carac == "," and nbr_parenthese == 0:
            if service.strip() != "":
                services_L.append(service.strip())
                service = ""

        else:
            service = service + carac

    if service.strip() != "":
        services_L.append(service.strip())

    return services_L

def Separer_Jour(texte):
    jours_L = []

    for jour in dico_horaires:
        if jour in texte:
            jours_L.append(jour)

    return jours_L # cette fonction va enfaite nous permettre de
    récupérer les jours où la station est ouverte

# cette fonction permet d'écrire l'heure sous la forme
demandée par exemple : 20.00 devient 20 et 20.30 reste 20.30
def Corriger_Heure(heure):
    heure = changer(heure)

    if heure == "":
        return ""

    heure = heure.replace(":", ".")
    morceaux = heure.split(".")

    if len(morceaux) == 2:
        h = morceaux[0]
        m = morceaux[1]

        if h.startswith("0") and len(h) > 1:
            h = h[1:]

        if m == "00":

```

```

        return h

    return h + "." + m

return heure

#cette fonction récupère les horaires d'un jour dans un texte
par exemple : Lundi 09.00-12.10 et 14.45-19.10
def Recuperer_Horaires_Du_Jour(partie_jour):
    horaires_L = []
    mot = ""
    lecture = False

    for carac in partie_jour:
        if carac.isdigit() or carac == "." or carac == ":" or
carac == "-":
            mot = mot + carac
            lecture = True

        else:
            if lecture == True:
                if "-" in mot:
                    horaires_L.append(mot)
                    mot = ""
                    lecture = False

    if mot != "":
        if "-" in mot:
            horaires_L.append(mot)

    return horaires_L

# cet fonction récupère les jours avec leurs heures
d'ouverture et de fermeture
def Separer_Horaire(texte, automate):
    resultat = []

    texte = changer(texte)
    automate = changer(automate)

```

```

    # si la station est en automate 24-24 et qu'il n'y a pas
    d'horaires détaillés alors on considère que chaque jour est
    ouvert de 00.00 à 23.59
    if texte == "Automate-24-24" or texte == "":
        if automate == "Oui" or "Automate-24-24" in texte:
            for jour in dico_horaires:
                resultat.append((jour, "0", "23.59"))
            return resultat

positions = []

# on cherche la position de chaque jour dans le texte
for jour in dico_horaires:
    position = texte.find(jour)

    if position != -1:
        positions.append((position, jour))

positions.sort()

# on découpe le texte jour par jour
for i in range(len(positions)):
    position_debut = positions[i][0]
    jour = positions[i][1]

    if i + 1 < len(positions):
        position_fin = positions[i + 1][0]
    else:
        position_fin = len(texte)

    partie_jour = texte[position_debut:position_fin]

    horaires_trouves =
Recuperer_Horaires_Du_Jour(partie_jour)

    for horaire in horaires_trouves:
        morceaux = horaire.split("-")

        if len(morceaux) == 2:
            heure_ouverture = Corriger_Heure(morceaux[0])
            heure_fermeture = Corriger_Heure(morceaux[1])

            if heure_ouverture != "" and heure_fermeture !=
"":

```

```

                                resultat.append((jour, heure_ouverture,
heure_fermeture))

    return resultat

```

```

temp = [
    "id",
    "latitude",
    "longitude",
    "code_postal",
    "pop",
    "adresse",
    "ville",
    "services",
    "prix",
    "rupture",
    "horaires",
    "geom",

    "prixGMJDATE",
    "prixG",

    "prixSP95MJDATE",
    "prixSP95",

    "prixE85MJDATE",
    "prixE85",

    "prixGPLcMJDATE",
    "prixGPLc",

    "prixE10MJDATE",
    "prixE10",

    "prixSP98MJDATE",
    "prixSP98",

    "ruptureE10",
    "typeRuptureE10",

    "ruptureSP98",
    "typeRuptureSP98",

```

```

    "ruptureSP95",
    "typeRuptureSP95",

    "ruptureE85",
    "typeRuptureE85",

    "ruptureGPLc",
    "typeRuptureGPLc",

    "ruptureGazole",
    "typeRuptureGazole",

    "carburantDispo",
    "carburantIndispo",
    "carburantRuptureTemp",
    "carburantRuptureDef",

    "automate",
    "servicesProposes",
    "departement",
    "codeDepartement",
    "region",
    "codeRegion",
    "horaireDetails"
]

```

```

dico_services = {} # ce dico va nous permettre d'associer
chaque nom de service à un id unique exemple : "Toilettes
publiques" -> 1

```

```

associations = [] #cet liste qui va contenir les liens entre
les stations et les services

```

```

dico_horaires = {
    "Lundi": 1,
    "Mardi": 2,
    "Mercredi": 3,
    "Jeudi": 4,
    "Vendredi": 5,
    "Samedi": 6,
    "Dimanche": 7
} # ce dico associe chaque jour à un id unique

```

```

# ce dico va permettre d'associer un id unique à chaque
horaire complet
# exemple : ("Lundi", "8", "20") -> 1
dico_horaires_id = {}

associations_horaires = [] # cet liste va contenir les liens
entre les stations et les horaires

nbr_ligne = 0

fichier_csv = open(debut, "r", encoding="utf-8-sig",
newline="")

lecteur = csv.DictReader(fichier_csv, delimiter=";")#
DictReader permet de lire chaque ligne comme un dico

sortie_prix = open(fichier_prix, "w", encoding="utf-8",
newline="")

ecriture_prix = csv.DictWriter(
    sortie_prix,
    fieldnames=temp,
    delimiter=";"
)# DictWriter permetta permettre d'écrire un CSV avec des noms
de colonnes bien précis

ecriture_prix.writeheader() #écrire la première ligne du
fichier csv

# On parcours de chaque ligne du fichier csv
for ligne in lecteur:

    nbr_ligne = nbr_ligne + 1

    # On corrige les problèmes avec les fonctions changer() et
correction() avant de les écrire.
    nouvelle_ligne = {
        "id": changer(ligne["id"]),
        "latitude": correction(ligne["latitude"]),
        "longitude": correction(ligne["longitude"]),

```

```

"code_postal": changer(ligne["Code postal"]),
"pop": changer(ligne["pop"]),
"adresse": changer(ligne["Adresse"]),
"ville": changer(ligne["Ville"]),
"services": changer(ligne["services"]),
"prix": changer(ligne["prix"]),
"rupture": changer(ligne["rupture"]),
"horaires": changer(ligne["horaires"]),
"geom": changer(ligne["geom"]),

# Gazole
"prixGMJDATE": changer(ligne["Prix Gazole mis à jour
le"]),
"prixG": correction(ligne["Prix Gazole"]),

# SP95
"prixSP95MJDATE": changer(ligne["Prix SP95 mis à jour
le"]),
"prixSP95": correction(ligne["Prix SP95"]),

# E85
"prixE85MJDATE": changer(ligne["Prix E85 mis à jour
le"]),
"prixE85": correction(ligne["Prix E85"]),

# GPLc
"prixGPLcMJDATE": changer(ligne["Prix GPLc mis à jour
le"]),
"prixGPLc": correction(ligne["Prix GPLc"]),

# E10
"prixE10MJDATE": changer(ligne["Prix E10 mis à jour
le"]),
"prixE10": correction(ligne["Prix E10"]),

# SP98
"prixSP98MJDATE": changer(ligne["Prix SP98 mis à jour
le"]),
"prixSP98": correction(ligne["Prix SP98"]),

# E10
"ruptureE10": changer(ligne["Début rupture e10 (si
temporaire)"]),
"typeRuptureE10": changer(ligne["Type rupture e10"]),

```



```

    # SP98
    "ruptureSP98": changer(ligne["Début rupture sp98 (si
temporaire)"]),
    "typeRuptureSP98": changer(ligne["Type rupture sp98"]),

    # SP95
    "ruptureSP95": changer(ligne["Début rupture sp95 (si
temporaire)"]),
    "typeRuptureSP95": changer(ligne["Type rupture sp95"]),

    # E85
    "ruptureE85": changer(ligne["Début rupture e85 (si
temporaire)"]),
    "typeRuptureE85": changer(ligne["Type rupture e85"]),

    # GPLc
    "ruptureGPLc": changer(ligne["Début rupture GPLc (si
temporaire)"]),
    "typeRuptureGPLc": changer(ligne["Type rupture GPLc"]),

    # Gazole
    "ruptureGazole": changer(ligne["Début rupture gazole
(si temporaire)"]),
    "typeRuptureGazole": changer(ligne["Type rupture
gazole"]),

    "carburantDispo": changer(ligne["Carburants
disponibles"]),
    "carburantIndispo": changer(ligne["Carburants
indisponibles"]),
    "carburantRuptureTemp": changer(ligne["Carburants en
rupture temporaire"]),
    "carburantRuptureDef": changer(ligne["Carburants en
rupture definitive"]),
    "automate": changer(ligne["Automate 24-24 (oui/non)"]),
    "servicesProposes": changer(ligne["Services
proposés"]),
    "departement": changer(ligne["Département"]),
    "codeDepartement": changer(ligne["code_departement"]),
    "region": changer(ligne["Région"]),
    "codeRegion": changer(ligne["code_region"]),
    "horaireDetails": changer(ligne["horaires détaillés"])

```

```

}

ecriture_prix.writerow(nouvelle_ligne)

# On récupère l'Id de la station
id_station = nouvelle_ligne["id"]

# On récupère les services qui sont proposées par une
station
texte_services = nouvelle_ligne["servicesProposes"]

# On sépare les différents services.
liste_services = Separer(texte_services)

services_temp = []
# on parcourt ici tous les services trouvés pour la station
actuelle, afin créer un id pour chaque service différent, puis
de créer le lien entre la station et ce service

for service in liste_services:

    service = changer(service)

    if service != "":

        if service not in dico_services:
            nouvel_id = len(dico_services) + 1
            dico_services[service] = nouvel_id
            id_service = dico_services[service]

        association = (id_station, id_service)

        if association not in services_temp :
            associations.append(association)
            services_temp.append(association)

# On récupère les horaires détaillés de la station
texte_horaire = nouvelle_ligne["horaireDetails"]

# on récupère les horaires avec le jour, l'heure
d'ouverture et l'heure de fermeture

```

```

    liste_horaires = Separer_Horaire(texte_horaire,
nouvelle_ligne["automate"])

    horaires_temp = []

    # on parcourt ici tous les horaires trouvés pour la station
actuelle
    for horaire in liste_horaires:

        jour = horaire[0]
        heure_ouverture = horaire[1]
        heure_fermeture = horaire[2]

        cle_horaire = (jour, heure_ouverture, heure_fermeture)

        if cle_horaire not in dico_horaires_id:
            nouvel_id = len(dico_horaires_id) + 1
            dico_horaires_id[cle_horaire] = nouvel_id

        id_horaire = dico_horaires_id[cle_horaire]

        association_horaire = (id_station, id_horaire)

        if association_horaire not in horaires_temp:
            associations_horaires.append(association_horaire)
            horaires_temp.append(association_horaire)

sortie_prix.close()
fichier_csv.close()

sortie_services = open(fichier_services, "w",
encoding="utf-8", newline="")

ecriture_services = csv.writer(sortie_services, delimiter=";")

ecriture_services.writerow(["id_service", "type_service"])

# On parcourt tous les services enregistrés dans le dico, afin
de mettre chaque service une seule fois dans le fichier
services.csv
for service in dico_services:
    id_service = dico_services[service]
    ecriture_services.writerow([id_service, service])

```

```

sortie_services.close()

sortie_station_services = open(fichier_station_services, "w",
encoding="utf-8", newline="")
ecriture_station_services =
csv.writer(sortie_station_services, delimiter=";")

ecriture_station_services.writerow(["id_station",
"id_service"])
# on parcourt toutes les associations station-service créées
et on fait en sorte que chaque association ait un id de la
station et l'id du service
for association in associations:
    id_station = association[0]
    id_service = association[1]

    ecriture_station_services.writerow([id_station,
id_service])
sortie_station_services.close()

sortie_horaires = open(fichier_horaires, "w",
encoding="utf-8", newline="")
ecriture_horaires = csv.writer(sortie_horaires, delimiter=";")

# maintenant le fichier horaires.csv contient l'id de
l'horaire, le jour, l'heure d'ouverture et l'heure de
fermeture
ecriture_horaires.writerow(["id_horaire", "jour",
"heure_ouverture", "heure_fermeture"])

#on parcourt tous les horaires enregistrés dans le dico, afin
de mettre chaque horaire une seule fois dans le fichier
horaires.csv
for cle_horaire in dico_horaires_id:
    id_horaire = dico_horaires_id[cle_horaire]
    jour = cle_horaire[0]
    heure_ouverture = cle_horaire[1]
    heure_fermeture = cle_horaire[2]

    ecriture_horaires.writerow([id_horaire, jour,
heure_ouverture, heure_fermeture])

```

```

sortie_horaires.close()

sortie_station_horaires = open(fichier_station_horaires, "w",
encoding="utf-8", newline="")
ecriture_station_horaires =
csv.writer(sortie_station_horaires, delimiter=";")

# maintenant station_horaires.csv contient id_station et
id_horaire
ecriture_station_horaires.writerow(["id_station",
"id_horaire"])

# on parcourt toutes les associations station-horaire créées
et on écrit l'id de la station avec l'id de l'horaire
correspondant
for association_horaire in associations_horaires:
    id_station = association_horaire[0]
    id_horaire = association_horaire[1]

    ecriture_station_horaires.writerow([id_station,
id_horaire])

sortie_station_horaires.close()

```